

તમે મોકલેલો **Node.js સિલેબસનો ફોટો** મને મળી ગયો છે. આમાં આપેલા દરેક મોડ્યુલ અને તેના તમામ કન્સેપ્ટને લાઇન બાય લાઇન, કોઈપણ ટોપિક મિસ કર્યા વગર એકદમ સરળ અને સાદી ગુજરાતી ભાષામાં વિગતવાર સમજાવે.

આપણે સીક્વન્સ પ્રમાણે **Module 1** થી શરૂઆત કરીએ.

---

## Module 1: Introduction to Node.js (નોડ.જીએસ નો પરિચય)

### ૧. What is Node.js (Node.js શું છે?)

- **સમજૂતી:** Node.js એ કોઈ પ્રોગ્રામિંગ લેંગ્વેજ (ભાષા) નથી અને કોઈ ફ્રેમવર્ક પણ નથી. તે જાવાસ્ક્રિપ્ટનું એક **Runtime Environment** (રનટાઇમ એન્વાયરમેન્ટ) છે.
- પહેલાના સમયમાં જાવાસ્ક્રિપ્ટ કોડ ફક્ત બ્રાઉઝરની અંદર જ રન થઈ શકતો હતો (જેમ કે કોમ કે ફાયરફોક્સમાં). પરંતુ, Node.js ના આવવાથી આપણે જાવાસ્ક્રિપ્ટ કોડને બ્રાઉઝરની બહાર, આપણા કમ્પ્યુટર અથવા સર્વર ઉપર સીધો રન કરી શકીએ છીએ.
- તે ગૂગલ કોમના પાવરફુલ **V8 JavaScript Engine** પર બનેલું છે. આ એન્જિન જાવાસ્ક્રિપ્ટ કોડને મશીન કોડમાં ફાસ્ટ કન્વર્ટ કરે છે, જેથી બેકએન્ડના કામો ખૂબ જ ઝડપથી થાય છે.

### ૨. Features of Node.js (Node.js ની મુખ્ય ખાસિયતો)

- **Asynchronous અને Non-blocking:** Node.js ની બધી જ API અસિન્ક્રોનસ છે. આનો અર્થ એ થાય કે જો કોઈ એક કામ પૂરું થવામાં વાર લાગતી હોય (જેમ કે ફાઇલ રીડ કરવી), તો Node.js તેના માટે અટકી નથી જતું, પણ બીજા કામો ચાલુ કરી દે છે.
- **ખૂબ જ ઝડપી (Very Fast):** V8 એન્જિનના કારણે તેનો કોડ એક્ઝિક્યુટ થવાનો સમય ખૂબ જ ઓછો છે.
- **Single-Threaded:** તે એક જ થ્રેડ (Single Thread) પર કામ કરે છે, પણ Event Loop ના કારણે એકસાથે હજારો રિક્વેસ્ટ હેન્ડલ કરી શકે છે.
- **કોઈ બફરિંગ નહીં (No Buffering):** Node.js માં ડેટા ક્યારેય બ્લોક કે બફર નથી થતો, તે ડેટાને સીધો ટુકડાઓમાં (Streams માં) આઉટપુટ આપે છે.

### ૩. Concept (તેનો મુખ્ય કન્સેપ્ટ)

- Node.js નો મુખ્ય સિદ્ધાંત "**Write Once, Run Everywhere**" અને જાવાસ્ક્રિપ્ટ ડેવલપર્સ માટે બેકએન્ડના દરવાજા ખોલવાનો છે.
- આના લીધે ફ્રન્ટએન્ડ ડેવલપર પણ સરળતાથી ફ્રન્ટ-સ્ટેક ડેવલપર બની શકે છે કારણ કે ફ્રન્ટએન્ડ અને બેકએન્ડ બંનેમાં એક જ લેંગ્વેજ (JavaScript) વપરાય છે.

### ૪. Where to fit and not fit (ક્યાં વાપરવું અને ક્યાં ન વાપરવું?)

- **ક્યાં વાપરવું (Where to fit):**
  - I/O Bound એપ્લિકેશન્સમાં (જ્યાં ડેટાની લેવડ-દેવડ વધારે હોય).
  - રીયલ-ટાઇમ ચેટ એપ્લિકેશન્સમાં (જેમ કે WhatsApp ક્લોન).

- ડેટા સ્ટ્રીમિંગ એપ્લિકેશન્સમાં (જેમ કે Netflix જેવી સાઇટ્સ).
- Single Page Applications (SPA) અને REST API બનાવવામાં.
- **ક્યાં ન વાપરવું (Where NOT to fit):**
  - CPU Bound એપ્લિકેશન્સમાં. એટલે કે જ્યાં ભારે ગણતરીઓ કરવાની હોય, વીડિયો એડિટિંગ કરવાનું હોય કે ઇમેજ પ્રોસેસિંગ કરવાનું હોય ત્યાં Node.js અનુકૂળ નથી, કારણ કે તે સિંગલ-થ્રેડેડ હોવાથી મોટી ગણતરી વખતે આખું સર્વર બ્લોક થઈ શકે છે.

#### ૫. Event-Driven programming style (ઇવેન્ટ-ડ્રિવન પ્રોગ્રામિંગ સ્ટાઇલ)

- **સમજૂતી:** ઇવેન્ટ-ડ્રિવનનો અર્થ એવો થાય કે એપ્લિકેશનમાં જ્યારે કોઈ ઇવેન્ટ (ઘટના) બને, ત્યારે જ તેનો કોડ રન થાય.
- દાખલા તરીકે: જ્યારે કોઈ યુઝર બટન પર "ક્લિક" કરે, કોઈ ફાઇલ ઓપન થાય અથવા સર્વર પર નવી રિક્વેસ્ટ આવે, ત્યારે Node.js એક ચોક્કસ ફંક્શન (જેને Callback ફંક્શન કહેવાય છે) ટ્રિગર કરે છે. આખી એપ્લિકેશન ઇવેન્ટ્સ અને લિસનર્સ (Listeners) ના આધારે જ કામ કરે છે.

#### ૬. What is Asynchronous (અસિન્ક્રોનસ એટલે શું?)

- **સમજૂતી:** સાદા પ્રોગ્રામિંગમાં (Synchronous) કોડ લાઇન બાય લાઇન ચાલે છે. જો લાઇન ૨ માં મોટું કામ હોય, તો લાઇન ૩ ત્યાં સુધી અટકેલી રહે છે.
- પરંતુ **Asynchronous** માં એવું નથી હોતું. જો કોઈ કામ પૂરું થવામાં સમય લેવાનું હોય (જેમ કે ડેટાબેઝમાંથી મોટો ડેટા લાવવો), તો Node.js તે કામ બેકગ્રાઉન્ડમાં ચાલુ કરી દે છે અને નીચેની લાઇનોનો કોડ રન કરવાનું શરૂ કરી દે છે. જેવું પહેલું કામ પૂરું થાય, એટલે ઇવેન્ટ લૂપ દ્વારા તેનો જવાબ પાછો મળી જાય છે. આનાથી યુઝરને ક્યારેય સાઇટ સ્લો હોવાનો અનુભવ નથી થતો.

### Module 2: Installation / Setup (ઇન્સ્ટોલેશન અને સેટઅપ)

#### ૧. Local environment setup (લોકલ એન્વાયરમેન્ટ સેટઅપ)

- Node.js માં કામ કરવા માટે તમારે તમારા કમ્પ્યુટરને તૈયાર કરવું પડે. આના માટે બે મુખ્ય વસ્તુઓ જોઈએ:
  1. Node.js નું સેટઅપ (Runtime).
  2. એક ટેક્સ્ટ એડિટર (જેમ કે VS Code - Visual Studio Code).

#### ૨. Node.js runtime (Node.js રનટાઇમ)

- રનટાઇમ એટલે એવો સોફ્ટવેર જે તમારા જાવાસ્ક્રિપ્ટ કોડને કમ્પ્યુટર સમજી શકે તેવા મશીન કોડમાં બદલીને રન કરવાની સગવડ આપે છે. Node.js ઇન્સ્ટોલ કરવાથી આ રનટાઇમ ઓટોમેટિક સેટ થઈ જાય છે.

#### ૩. Download source code (સોર્સ કોડ ડાઉનલોડ કરવો)

- Node.js ડાઉનલોડ કરવા માટે તેની ઓફિશિયલ વેબસાઇટ **nodejs.org** પર જવું પડે છે. ત્યાં બે વર્ઝન જોવા મળે છે:

- **LTS (Long Term Support):** આ વર્ઝન એકદમ સ્ટેબલ (સ્થિર) હોય છે અને પ્રોજેક્ટ્સ માટે આ જ ડાઉનલોડ કરવું બેસ્ટ ગણાય છે.
- **Current:** આમાં નવા ફીચર્સ હોય છે પણ તે ટેસ્ટિંગ ફેઝમાં હોઈ શકે છે.

#### ૪. Installation on OS (ઓપરેટિંગ સિસ્ટમ પર ઇન્સ્ટોલેશન)

- **Windows/Mac માટે:** વેબસાઇટ પરથી સાદી .msi અથવા .pkg ફાઇલ ડાઉનલોડ કરીને માત્ર "Next", "Next" અને "Finish" આપવાથી સોફ્ટવેર ઇન્સ્ટોલ થઈ જાય છે.
- **Linux માટે:** ટર્મિનલની અંદર પેકેજ મેનેજર (જેમ કે apt કે yum) ના કમાન્ડ્સ લખીને ઇન્સ્ટોલ કરવામાં આવે છે.

#### ૫. Verify (વેરિફિકેશન - ચેક કરવું)

- Node.js બરાબર ઇન્સ્ટોલ થયું છે કે નહીં તે ચેક કરવા માટે તમારા કમ્પ્યુટરનું ટર્મિનલ અથવા કમાન્ડ પ્રોમ્પ્ટ (CMD) ઓપન કરીને નીચેના કમાન્ડ લખવામાં આવે છે:
  - node -v (આનાથી Node નું વર્ઝન દેખાશે).
  - npm -v (આનાથી NPM નું વર્ઝન દેખાશે).
- જો વર્ઝન નંબર સ્ક્રીન પર આવી જાય, એનો અર્થ એ કે સેટઅપ સફળ થઈ ગયું છે.

---

### Module 3: Node Package Manager - NPM (નોડ પેકેજ મેનેજર)

#### ૧. Install module by NPM (NPM દ્વારા મોડ્યુલ ઇન્સ્ટોલ કરવા)

- **NPM શું છે?:** NPM એ દુનિયાનું સૌથી મોટું સોફ્ટવેર રજિસ્ટ્રી પ્લેટફોર્મ છે. બીજા ડેવલપર્સે બનાવેલા તૈયાર કોડના ટુકડા (જેને આપણે Packages કે Modules કહીએ છીએ) વાપરવા માટે NPM નો ઉપયોગ થાય છે.
- મોડ્યુલ ઇન્સ્ટોલ કરવા માટેનો સાદો કમાન્ડ:  
npm install package\_name (દાખલા તરીકે: npm install express).

#### ૨. Global vs Local setup (ગ્લોબલ વર્સિસ લોકલ સેટઅપ)

- **Local Installation:** જ્યારે આપણે સાદું npm install કરીએ ત્યારે તે મોડ્યુલ ફક્ત આપણા એ જ ચોક્કસ પ્રોજેક્ટના ફોલ્ડરમાં (node\_modules ની અંદર) ઇન્સ્ટોલ થાય છે. તે બીજા પ્રોજેક્ટમાં વાપરી શકાતું નથી.
- **Global Installation:** જો કોઈ એવો ટૂલ હોય જે આખા કમ્પ્યુટરમાં ગમે ત્યાંથી વાપરવો હોય, તો તેની પાછળ -g ફ્લેગ લગાવાય છે.
  - કમાન્ડ: npm install -g package\_name (જેમ કે npm install -g nodemon).

#### ૩. Update Module (મોડ્યુલ અપડેટ કરવું)

- સમય જતાં જે તે પેકેજમાં નવા સુધારા આવે ત્યારે તેને અપડેટ કરવા માટે નીચેનો કમાન્ડ વપરાય છે:  
npm update package\_name
- આ કમાન્ડ પ્રોજેક્ટની package.json ફાઇલમાં ચેક કરીને લેટેસ્ટ વર્ઝન ડાઉનલોડ કરી લે છે.

## ૪. CRUD Module (મોડ્યુલનું મેનેજમેન્ટ - CRUD ઓપરેશન્સ)

- NPM ની મદદથી આપણે પ્રોજેક્ટના પેકેજનું લિસ્ટ મેનેજ કરી શકીએ છીએ:
  - **Create:** npm init કમાન્ડથી નવો પ્રોજેક્ટ સેટ કરવો અને નવી package.json ફાઇલ બનાવવી.
  - **Read:** npm list કમાન્ડથી પ્રોજેક્ટમાં કયા-કયા મોડ્યુલ ઇન્સ્ટોલ છે તે જોવું.
  - **Update:** npm update થી મોડ્યુલ બદલવું.
  - **Delete (Uninstall):** npm uninstall package\_name કમાન્ડથી બિનજરૂરી મોડ્યુલને પ્રોજેક્ટમાંથી કાયમ માટે હટાવી દેવું.

---

મેં સિલેબસના **Module 1, 2 અને 3** ના તમામ પોઇન્ટ્સ એકદમ સરળ ગુજરાતીમાં સમજાવી દીધા છે. આગળના **Module 4 (Express Framework)** પર જઈએ? જો આ ત્રણ મોડ્યુલમાં કોઈ પ્રશ્ન હોય તો પહેલા પૂછી શકો છો!

ચાલો, હવે આપણે આગળ વધીએ અને **Module 4 (Express Framework)**

તથા તેની પછીના મોડ્યુલ્સને એકદમ વિગતવાર અને સાદી ગુજરાતીમાં સમજાવે.

જ્યાં કોડ કે સિન્ટેક્સની જરૂર છે, ત્યાં સરળ ઉદાહરણો પણ સાથે આપેલા છે જેથી તમને સમજવામાં એકદમ સરળતા રહે.

---

## Module 4: Express Framework (એક્સપ્રેસ ફ્રેમવર્ક) [1]

### ૧. What is Express (Express શું છે?) [1]

- **સમજૂતી:** Node.js માં સર્વર અને વેબ એપ્લિકેશન બનાવવા માટે આપણે ઇન-બિલ્ટ http મોડ્યુલ વાપરવું પડે છે, જેમાં ઘણો બધો મોટો કોડ લખવો પડે છે. **Express.js** એ Node.js માટેનું એક નાનું, ઝડપી અને મિનિમલિસ્ટ (Minimalist) વેબ ફ્રેમવર્ક છે.
- તે Node.js ની ઉપર એક લેયર તરીકે કામ કરે છે, જે વેબ સર્વર, API, અને રાઉટિંગ (Routing) બનાવવાનું કામ ખૂબ જ સરળ અને ટૂંકું કરી દે છે.

### ૨. Setup Express (Express સેટઅપ કરવું) [1]

- તમારા પ્રોજેક્ટ ફોલ્ડરમાં એક્સપ્રેસ ઇન્સ્ટોલ કરવા માટે નીચેનો કમાન્ડ રન કરાવ છે:  
npm install express
- **બેઝિક સર્વર બનાવવાનો કોડ:**

javascript

```
const express = require('express');
```

```
const app = express();
```

```
app.listen(3000, () => {  
  console.log("Server 3000 પોર્ટ પર ચાલુ થઈ ગયું છે.");  
});
```

Use code with caution.

### ૩. Request And Response (રિક્વેસ્ટ અને રિસ્પોન્સ) [1]

- જ્યારે પણ કોઈ ક્લાયન્ટ (જેમ કે બ્રાઉઝર) આપણા સર્વરને કોઈ વિનંતી મોકલે, ત્યારે તેને **Request (req)** કહેવાય છે. આમાં યુઝરનો ડેટા, URL અને હેડર્સ હોય છે.
- સર્વર એ રિક્વેસ્ટ પ્રોસેસ કરીને જે વળતો જવાબ બ્રાઉઝરને મોકલે છે, તેને **Response (res)** કહેવાય છે.
- ઉદાહરણ:** `res.send("હેલો યુઝર!");` ની મદદથી આપણે બ્રાઉઝર પર લખાણ મોકલી શકીએ છીએ.

### ૪. Handling Routes (રાઉટ્સ હેન્ડલ કરવા) [1]

- રાઉટીંગ એટલે શું?:** વેબસાઇટ પર અલગ-અલગ પેજ્સ કે URL ઓપન કરવા પર શું કામ થવું જોઈએ તે નક્કી કરવું.
- ઉદાહરણ:**

javascript

```
// હોમ પેજ માટેનું રાઉટ  
app.get('/', (req, res) => {  
  res.send("આ હોમ પેજ છે.");  
});
```

```
// અબાઉટ પેજ માટેનું રાઉટ  
app.get('/about', (req, res) => {  
  res.send("આ અમારા વિશેનું પેજ છે.");  
});
```

Use code with caution.

### ૫. Route Middleware (રાઉટ મિડલવેર) [1]

- સમજૂતી:** મિડલવેર એવા ફંક્શન્સ છે જે યુઝરની રિક્વેસ્ટ (Request) સર્વર સુધી પહોંચે તે પહેલાં વચ્ચે (Middle માં) રન થાય છે.
- તેનો ઉપયોગ યુઝર લોગઇન છે કે નહીં (Authentication) તે ચેક કરવા માટે અથવા રિક્વેસ્ટનો લોગ રાખવા માટે થાય છે. તેમાં `req`, `res` અને એક `next()` ફંક્શન હોય છે, જે રિક્વેસ્ટને આગળના કોડ તરફ ધકેલે છે.

### ૬. Objects (ઓબ્જેક્ટ્સ) [1]

- Express માં મુખ્ય બે ઓબ્જેક્ટ્સ સૌથી વધારે વપરાય છે:
  - req.query:** URL માંથી ક્વેરી પેરામીટર્સ વાંચવા માટે (જેમ કે /search?name=rahul).
  - req.params:** URL માંથી ડાયનેમિક આઈડી વાંચવા માટે (જેમ કે /user/:id).
  - req.body:** ફોર્મ સબમિટ કરતી વખતે મોકલેલો ડેટા વાંચવા માટે.

## 9. Cookies Management (કુકીઝ મેનેજમેન્ટ) [1]

- બ્રાઉઝરની અંદર નાની માહિતી (જેમ કે યુઝરનું સેશન) સ્ટોર કરવા માટે કુકીઝ વપરાય છે.
- Express માં કુકીઝ વાંચવા અને સેટ કરવા માટે cookie-parser નામના મોડ્યુલનો ઉપયોગ થાય છે.
  - કુકી સેટ કરવા: `res.cookie('username', 'rahul');`
  - કુકી ડીલીટ કરવા: `res.clearCookie('username');`

## ૮. File upload (ફાઇલ અપલોડ) [1]

- યુઝર પ્રોફાઇલ પિક્ચર, PDF કે કોઈ ડોક્યુમેન્ટ સર્વર પર અપલોડ કરે ત્યારે તેને હેન્ડલ કરવા માટે Express માં **multer** નામના મિડલવેર મોડ્યુલનો ઉપયોગ કરવામાં આવે છે. તે ફાઇલને સુરક્ષિત રીતે સર્વરના ફોલ્ડરમાં સેવ કરે છે.

## ૯. HTTP methods (HTTP મેથડ્સ) [1]

- API બનાવતી વખતે ડેટાની લેવડ-દેવડ માટે 4 મુખ્ય HTTP મેથડ્સ વપરાય છે:
  - GET:** સર્વર પરથી ડેટા મેળવવા (વાંચવા) માટે.
  - POST:** સર્વર પર નવો ડેટા મોકલવા (ક્રિએટ કરવા) માટે.
  - PUT:** સર્વર પર રહેલા જૂના ડેટાને બદલવા (અપડેટ કરવા) માટે.
  - DELETE:** સર્વર પરથી ડેટા કાઢી નાખવા માટે.

## Module 5: Buffers and Streams (બફર્સ અને સ્ટ્રીમ્સ) [1]

### ૧. What is buffers and Streams (બફર્સ અને સ્ટ્રીમ્સ શું છે?) [1]

- Buffer:** જ્યારે બાયનરી ડેટા (૦ અને ૧ ના ફોર્મમાં રહેલો ડેટા) એક જગ્યાએથી બીજી જગ્યાએ ટ્રાન્સફર થતો હોય, ત્યારે તેને ટેમ્પરરી સ્ટોર કરવા માટે કમ્પ્યુટરની મેમરીમાં (RAM માં) જે નાની જગ્યા બને છે તેને **Buffer** કહેવાય છે.
- Stream:** ખૂબ જ મોટા ડેટાને (જેમ કે મોટી વીડિયો કે ઓડિયો ફાઇલ) એકસાથે લોડ કરવાને બદલે તેના નાના-નાના ટુકડા કરીને સતત મોકલવાની કે વાંચવાની પ્રક્રિયાને **Stream** કહેવાય છે.

### ૨. Benefit of Streams over buffers (બફર્સ કરતાં સ્ટ્રીમ્સના ફાયદા) [1]

- જો આપણે ૧ GB ની ફાઇલ વાંચવા માટે **Buffer** વાપરીએ, તો તે આપણે ૧ GB ફાઇલ પહેલાં RAM માં લોડ થશે, જેનાથી આપણું કમ્પ્યુટર કે સર્વર સ્વો (હેંગ) થઈ શકે છે.

- પરંતુ જો આપણે **Stream** વાપરીએ, તો તે ફાઇલના નાના-નાના ટુકડા (દા.ત. 64 KB) કરીને પ્રોસેસ કરશે, જેનાથી ખૂબ જ ઓછી RAM વપરાશે અને કામ ઘણું ફાસ્ટ થશે. YouTube અને Netflix આ જ ટેકનોલોજી પર કામ કરે છે.

### 3. Create / Write / Read operation on Buffers (બફર્સ પર ઓપરેશન્સ) [1]

- Node.js માં ગ્લોબલ Buffer ક્લાસ આપેલો છે:
  - બફર બનાવવું: `const buf = Buffer.alloc(10);` (૧૦ બાઇટ્સનું ખાલી બફર).
  - બફરમાં ડેટા લખવો: `buf.write("Hello");`
  - બફરનો ડેટા વાંચવો: `console.log(buf.toString());`

### ૪. Process on Buffers (બફર્સ પર પ્રોસેસ) [1]

- આપણે બે બફરને જોડી શકીએ છીએ (`Buffer.concat([buf1, buf2])`), બે બફર સરખા છે કે નહીં તે સરખાવી શકીએ છીએ (`buf1.compare(buf2)`), અને બફરનો ડેટા કોપી પણ કરી શકીએ છીએ.

### ૫. Read / Write data by Streams (સ્ટ્રીમ્સ દ્વારા ડેટા રીડ/રાઇટ કરવો) [1]

- Node.js ની fs (File System) મોડ્યુલની મદદથી આપણે સ્ટ્રીમ બનાવી શકીએ છીએ:
  - ફાઇલ વાંચવા માટે: `const readStream = fs.createReadStream('largefile.txt');` [1]
  - ફાઇલમાં લખવા માટે: `const writeStream = fs.createWriteStream('output.txt');` [1]

### ૬. Pipeline (પાઇપલાઇન) [1]

- Pipeline** નો ઉપયોગ એક રીડ સ્ટ્રીમ માંથી ડેટા લઈને તેને રાઇટ સ્ટ્રીમમાં મોકલવા માટે થાય છે. આ પ્રક્રિયા દરમિયાન જો વચ્ચે કોઈ એરર (ભૂલ) આવે, તો પાઇપલાઇન તેને ઓટોમેટિક હેન્ડલ કરીને સ્ટ્રીમ્સને સુરક્ષિત રીતે બંધ કરી દે છે, જેથી મેમરી લીક થતી નથી.

### ૭. Chaining Stream (ચેનિંગ સ્ટ્રીમ) [1]

- એક સ્ટ્રીમના આઉટપુટને બીજી સ્ટ્રીમના ઇનપુટ સાથે જોડવાની પ્રક્રિયાને ચેનિંગ કહેવાય છે. દાખલા તરીકે: એક મોટી ટેક્સ્ટ ફાઇલને સ્ટ્રીમ દ્વારા રીડ કરવી, તેને વચ્ચે ઝીપ ફાઇલમાં કમ્પ્રેસ (Compress) કરવી અને પછી ફાઇલમાં રાઇટ કરવી.

## Module 6: REST API (રેસ્ટ એપીઆઈ) [1]

### ૧. RESTful Architecture (RESTful આર્કિટેક્ચર) [1]

- REST** નું પૂરું નામ **Representational State Transfer** છે. તે બે અલગ-અલગ સોફ્ટવેર કે એપ્લિકેશન (જેમ કે ફ્રન્ટએન્ડ મોબાઇલ એપ અને બેકએન્ડ સર્વર) વચ્ચે ડેટાની આપ-લે કરવા માટેના નિયમોનું એક આર્કિટેક્ચર માળખું છે. તે એકદમ સ્ટેટલેસ (Stateless) હોય છે, એટલે કે સર્વર દરેક રિક્વેસ્ટને તદ્દન નવી જ ગણે છે.

### ૨. HTTP URI and Methods (HTTP URI અને મેથડ્સ) [1]

- REST API માં દરેક ડેટા કે સોર્સ માટે એક અનોખું વેબ એડ્રેસ હોય છે જેને **URI (Uniform Resource Identifier)** કહેવાય છે, જેમ કે `/api/users`.

- આ URI ની સાથે યોગ્ય HTTP મેથડ્સ વાપરવી પડે છે:
  - GET /api/users -> બધા યુઝર્સનું લિસ્ટ મેળવવા.
  - POST /api/users -> નવો યુઝર ઉમેરવા.
  - DELETE /api/users/1 -> ૧ નંબરના યુઝરને કાઢી નાખવા.

### ૩. RESTful web services (RESTful વેબ સર્વિસીસ) [1]

- એવી વેબ સર્વિસ જે REST ના નિયમોનું પાલન કરીને બનાવવામાં આવી હોય અને મોટેભાગે ડેટાની આપ-વે કરવા માટે JSON (JavaScript Object Notation) ફોર્મેટનો ઉપયોગ કરતી હોય તેને RESTful વેબ સર્વિસ કહેવાય છે.

### ૪. Expose solution as API (સોલ્યુશનને API તરીકે એક્સપોઝ કરવું) [1]

- જ્યારે આપણે બેકએન્ડમાં કોઈ પ્રોગ્રામ કે ડેટાબેઝનું લોજિક બનાવીએ, અને તેને ફ્રન્ટએન્ડ ડેવલપર્સ વાપરી શકે તે માટે રાઉટ્સ (app.get, app.post) દ્વારા લિક સ્વરૂપે ખુલ્લું કરીએ, તેને સોલ્યુશનને API તરીકે એક્સપોઝ કર્યું કહેવાય.

### ૫. Best practice for REST API solution (શ્રેષ્ઠ પદ્ધતિઓ) [1]

- API ને હંમેશા વર્ઝન આપવું જોઈએ (જેમ કે /api/v1/users).
- ક્લેક્શનના નામ હંમેશા બહુવચનમાં રાખવા (જેમ કે /users, /products, નહિ કે /user).
- યોગ્ય HTTP Status Codes મોકલવા (દા.ત. સફળતા માટે 200 OK, નવો ડેટા બને તો 201 Created, અને એરર આવે તો 404 Not Found).

## Module 7: Callback (કોલબેક) [1]

### ૧. What is Callback (Callback શું છે?) [1]

- **સમજૂતી:** જાવાસ્ક્રિપ્ટમાં ફંક્શન પણ એક ઓબ્જેક્ટ છે. જ્યારે આપણે કોઈ એક ફંક્શનની અંદર આર્ગ્યુમેન્ટ (Argument) તરીકે બીજા ફંક્શનને પાસ કરીએ, અને પહેલું ફંક્શન પોતાનું કામ પૂરું કર્યા પછી એ બીજા ફંક્શનને રન કરે, ત્યારે એ બીજા ફંક્શનને **Callback ફંક્શન** કહેવાય છે.

### ૨. Benefit of Callback (Callback ના ફાયદા) [1]

- આનાથી આપણો કોડ અટકી (Block) નથી જતો. જ્યારે સર્વર કોઈ મોટું કામ કરતું હોય, ત્યારે બાકીનો કોડ ચાલુ રહે છે અને જેવું મોટું કામ પૂરું થાય કે તરત કોલબેક ફંક્શન તેનો જવાબ લઈને આવી જાય છે.

### ૩. Asynchronous communications (અસિન્ક્રોનસ કોમ્યુનિકેશન) [1]

- Node.js માં નેટવર્ક કે ફાઇલ સિસ્ટમ સાથે વાતચીત કરવા માટે કોલબેકનો ઉપયોગ થાય છે. સર્વર રિસ્પેસ્ટ મોકલીને જવાબની રાહ જોયા વગર બીજું કામ કરે છે, અને ડેટા આવી જાય ત્યારે કોલબેક દ્વારા વાતચીત પૂરી થાય છે.

### ૪. Block and Non-Blocking (બ્લોકિંગ અને નોન-બ્લોકિંગ) [1]

- **Blocking (Synchronous):** કોડ લાઇન બાય લાઇન ચાલે છે. જ્યાં સુધી પહેલી લાઇનનું ફાઇલ રીડિંગ પૂરું ન થાય ત્યાં સુધી આખું સર્વર બીજા યુઝર્સ માટે બ્લોક થઈ જાય છે.
- **Non-Blocking (Asynchronous):** કોડ ફાઇલ રીડિંગનું કામ બેકગ્રાઉન્ડમાં સોંપીને આગળ વધી જાય છે. સર્વર ચાલુ રહે છે (નોન-બ્લોકિંગ) અને કોલબેક દ્વારા ડેટા પાછો મળે છે.

#### ૫. Standard Callback pattern (સ્ટાન્ડર્ડ કોલબેક પેટર્ન) [1]

- Node.js માં એક નિયમ છે કે કોલબેક ફંક્શનની અંદર સૌથી પહેલો પેરામીટર હંમેશા **Error (err)** હોવો જોઈએ અને બીજો પેરામીટર **Data (result)** હોવો જોઈએ. આને **Error-First Callback** પેટર્ન કહે છે.

#### • ઉદાહરણ:

javascript

```
fs.readFile('data.txt', (err, data) => {
  if (err) return console.log("ભૂલ આવી!");
  console.log(data.toString());
});
```

Use code with caution.

#### ૬. Async Flow control Library (અસિંક ફ્લો કંટ્રોલ લાઇબ્રેરી) [1]

- જ્યારે એકની અંદર એક ઘણા બધા કોલબેક આવી જાય ત્યારે કોડ ખૂબ જ અઘરો બની જાય છે જેને **"Callback Hell"** કહેવાય છે. આ ફ્લોને કંટ્રોલ કરવા માટે **async** નામની એક્સટર્નલ લાઇબ્રેરી વાપરવામાં આવતી હતી (જોકે હવે આના બદલે નવું Promises અને **async/await** વપરાય છે).

#### ૭. Executing in parallel (પેરેલલ એક્ઝિક્યુશન) [1]

- જ્યારે આપણે એકસાથે ૩ કે ૪ અલગ-અલગ ફાઇલો વાંચવી હોય અથવા અલગ-અલગ API માંથી ડેટા લાવવો હોય, ત્યારે એક પછી એક કરવાના બદલે આપણે તેને **Parallel (એક જ સમયે એકસાથે)** રન કરી શકીએ છીએ જેથી સમય બચે છે.

### Module 8: Events (ઇવેન્ટ્સ) [1]

#### ૧. What is Events (Events શું છે?) [1]

- Node.js માં કોઈ ચોક્કસ પ્રવૃત્તિ કે ઘટના બને (જેમ કે સર્વર પર કોઈ કનેક્શન આવવું, ફાઇલ ઓપન થવી), તેને **Event** કહેવાય છે. Node.js નું આખું કોર આર્કિટેક્ચર ઇવેન્ટ્સ પર જ ચાલે છે.

#### ૨. Events types (ઇવેન્ટ્સના પ્રકાર) [1]

- **સિસ્ટમ ઇવેન્ટ્સ:** જે Node.js ના કોર મોડ્યુલ્સ દ્વારા ઓટોમેટિક જનરેટ થાય છે (દા.ત. સ્ટ્રીમમાં ડેટા આવે ત્યારે **data** ઇવેન્ટ, ફાઇલ પૂરી થાય ત્યારે **end** ઇવેન્ટ).
- **કસ્ટમ ઇવેન્ટ્સ:** જે આપણે પ્રોગ્રામર તરીકે આપણી જાતે બનાવીએ છીએ.

#### ૩. Event Emitter API (ઇવેન્ટ એમિટર API) [1]

- Node.js માં કસ્ટમ ઇવેન્ટ્સ બનાવવા અને તેને હેન્ડલ કરવા માટે **events** મોડ્યુલની અંદર **EventEmitter** ક્લાસ આપેલો છે.
  - ઇવેન્ટ ફાયર (ટ્રિગર) કરવા માટે: `emitter.emit('event_name');`
  - ઇવેન્ટ સાંભળવા (હેન્ડલ કરવા) માટે: `emitter.on('event_name', callback);`

#### ૪. Multiple event listeners (મલ્ટિપલ ઇવેન્ટ લિસનર્સ) [1]

- આપણે કોઈ એક જ ઇવેન્ટ માટે એક કરતાં વધારે લિસનર્સ (Listeners) ગોઠવી શકીએ છીએ. જ્યારે એ ઇવેન્ટ બનશે, ત્યારે તેની સાથે જોડાયેલા બધા જ ફંક્શન્સ ક્રમબદ્ધ રીતે એક પછી એક રન થશે.

#### ૫. Event emitter pattern (ઇવેન્ટ એમિટર પેટર્ન) [1]

- આ એક ડિઝાઇન પેટર્ન છે જેમાં ઓબ્જેક્ટ્સ સીધા એકબીજા સાથે વાત કરવાના બદલે ઇવેન્ટ્સ દ્વારા વાતચીત કરે છે. આનાથી મોટો કોડ મેનેજ કરવો એકદમ સરળ બની જાય છે.

#### ૬. Class methods (ક્લાસ મેથડ્સ) [1]

- EventEmitter ક્લાસમાં ઘણી મહત્વની મેથડ્સ છે:
  - `once()`: આ લિસનર ઇવેન્ટ બનવા પર ફક્ત એક જ વાર રન થશે, બીજી વાર ઇવેન્ટ બનશે તો રન નહીં થાય.
  - `removeListener()`: ચાલુ ઇવેન્ટમાંથી કોઈ લિસનરને હટાવવા માટે.

#### ૭. Event Loop (ઇવેન્ટ લૂપ) [1]

- સમજૂતી:** ઇવેન્ટ લૂપ એ Node.js નો સૌથી મુખ્ય આત્મા છે. તે બેકગ્રાઉન્ડમાં ચાલતું એક કન્ટીન્યુઅસ લૂપ (ચક્ર) છે જે સતત ચેક કરતું રહે છે કે કોઈ અસિન્ક્રોનસ કામ (જેમ કે ડેટાબેઝ ક્વેરી) પૂરું થયું છે કે નહીં. જેવું કામ પૂરું થાય, એટલે તે ક્યુ (Queue) માંથી કોલબેક ફંક્શનને ઉપાડીને મેઇન થ્રેડ પર રન કરવા માટે મોકલી આપે છે.

#### ૮. Blocking Event Loop (ઇવેન્ટ લૂપ બ્લોક થવું) [1]

- જો આપણે આપણા કોડમાં કોઈ ખૂબ જ ભારે લૂપ (દા.ત. ૧૦ કરોડ વખત ચાલતું for લૂપ) કે ભારે ગણતરી મૂકી દઈએ, તો Node.js નો સિંગલ મેઇન થ્રેડ તેમાં અટવાઈ જાય છે. આના કારણે ઇવેન્ટ લૂપ ફરી શકતું નથી અને આખું સર્વર બ્લોક થઈ જાય છે. બીજા નવા યુઝર્સ સાઇટ ઓપન કરી શકતા નથી.

#### ૯. Escaping Event Loop (ઇવેન્ટ લૂપમાંથી બચવું) [1]

- ઇવેન્ટ લૂપ બ્લોક ન થાય તે માટે ભારે ગણતરી વાળા કામોને નાના ટુકડાઓમાં વહેંચી દેવા જોઈએ, અથવા `setImmediate()` કે `process.nextTick()` નો ઉપયોગ કરીને તેને ઇવેન્ટ લૂપના આગલા ચક્ર પર ધકેલી દેવા જોઈએ. ખૂબ જ ભારે કામો માટે ચાઇલ્ડ પ્રોસેસ (Child Process) નો ઉપયોગ કરવો બેસ્ટ છે.

---

### Module 9: Connecting with Database (ડેટાબેઝ સાથે કનેક્શન) [1]

#### ૧. Introduction of MySQL (MySQL નો પરિચય) [1]

- **સમજૂતી:** MySQL એ એક ખૂબ જ લોકપ્રિય **Relational Database Management System (RDBMS)** છે. તે ડેટાને રો (Rows) અને કોલમ (Columns) ના સ્વરૂપમાં એટલે કે ટેબલ્સ (Tables) માં સ્ટોર કરે છે અને તેમાં કનેક્શન માટે SQL ક્વેરીઝ લખવી પડે છે.

## ૨. Connect with MySQL (MySQL સાથે કનેક્ટ કરવું) [1]

- Node.js માં MySQL વાપરવા માટે mysql2 નામનું પેકેજ ઇન્સ્ટોલ કરવું પડે છે.
- **કનેક્શન કોડ:**

javascript

```
const mysql = require('mysql2');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  database: 'test_db'
});

connection.query('SELECT * FROM users', (err, results) => {
  console.log(results);
});
```

Use code with caution.

## ૩. Introduction of MongoDB (MongoDB નો પરિચય) [1]

- **સમજૂતી:** MongoDB એ એક આધુનિક **NoSQL Database** છે. તે ટેબલ્સના બદલે ડેટાને **JSON જેવા ડોક્યુમેન્ટ્સ (BSON)** ના સ્વરૂપમાં સ્ટોર કરે છે. તે ફ્લેક્સિબલ હોય છે અને ખૂબ જ મોટા ડેટા હેન્ડલ કરવા માટે બેસ્ટ ગણાય છે.

## ૪. Features of MongoDB (MongoDB ની ખાસિયતો) [1]

- **Schema-less:** આમાં અગાઉથી કોઈ ફિક્સ બોર્ડર કે કોલમ નક્કી કરવાની જરૂર નથી હોતી, દરેક ડોક્યુમેન્ટ અલગ ડેટા સ્ટોર કરી શકે છે.
- **Scalability:** તે ખૂબ જ સરળતાથી મોટા સર્વિસ પર ડેટા વહેંચી શકે છે.

## ૫. Connect with MongoDB (MongoDB સાથે કનેક્ટ કરવું) [1]

- Node.js માં ઓર્ગેનાઇઝ્ડ રીતે મોંગોડીબી વાપરવા માટે **mongoose** નામના પ્રખ્યાત ઓબ્જેક્ટ મોડેલિંગ (ODM) પેકેજનો ઉપયોગ થાય છે.
- **કનેક્શન કોડ:**

javascript

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/my_db');
```

Use code with caution.

#### ૬. Defining a schema (સ્કીમા વ્યાખ્યાયિત કરવી) [1]

- મોંગોડીબી સ્કીમા-લેસ છે, છતાં આપણી એપ્લિકેશનમાં ભૂલ ન આવે તે માટે આપણે Mongoose ની મદદથી ડેટાનું એક ફોર્મેટ (માળખું) નક્કી કરીએ છીએ, તેને **Schema** કહેવાય છે.

- ઉદાહરણ:

javascript

```
const userSchema = new mongoose.Schema({  
  name: String,  
  age: Number,  
  email: String  
});
```

Use code with caution.

#### ૭. Defining a Model (મોડેલ વ્યાખ્યાયિત કરવું) [1]

- એકવાર સ્કીમા બની જાય, પછી તેના આધારે ડેટાબેઝના ક્વેક્શન સાથે વાતચીત કરવા (ડેટા સેવ, ડીલીટ કે અપડેટ કરવા) માટે જે ઓબ્જેક્ટ બને છે, તેને **Model** કહેવાય છે.

- ઉદાહરણ:

javascript

```
const User = mongoose.model('User', userSchema);  
  
// નવો યુઝર ડેટાબેઝમાં સેવ કરવા માટે Model નો ઉપયોગ:  
  
const newUser = new User({ name: "Rahul", age: 25 });  
  
newUser.save();
```

Use code with caution.

---

### Module 10: External Processes and Services (બાહ્ય પ્રક્રિયાઓ અને સેવાઓ) [1]

#### ૧. What is processes (પ્રોસેસ એટલે શું?) [1]

- જ્યારે આપણા કમ્પ્યુટરમાં કોઈ પણ પ્રોગ્રામ રન થઈ રહ્યો હોય, ત્યારે ઓપરેટિંગ સિસ્ટમ તેને જે મેમરી અને આઈડી આપે છે તેને **Process (પ્રક્રિયા)** કહેવાય છે. આપણે જે Node.js નો પ્રોગ્રામ રન કરીએ છીએ તે પણ કમ્પ્યુટરમાં એક પ્રોસેસ તરીકે જ ચાલે છે, જેને આપણે process ઓબ્જેક્ટથી કંટ્રોલ કરી શકીએ છીએ.

#### ૨. Spawning Child process (ચાઇલ્ડ પ્રોસેસ સ્પોન કરવી) [1]

- **સમજૂતી:** Node.js સિંગલ-થ્રેડેડ છે. તેથી જો કોઈ મોટું કે ભારે કામ કરવું હોય, તો Node.js પોતાના જેવી જ એક નવી સબ-પ્રોસેસ (બાળ પ્રક્રિયા) ઓપરેટિંગ સિસ્ટમમાં ચાલુ કરી શકે છે, જેને **Child Process** કહેવાય છે.
- આના માટે Node.js માં `child_process` મોડ્યુલની અંદર **`spawn()`** અથવા **`exec()`** મેથડ આપેલી છે. તે બેકગ્રાઉન્ડમાં કમ્પ્યુટરના બીજા કમાન્ડ્સ (જેમ કે લિનક્સ કમાન્ડ કે પાઇથોન સ્ક્રિપ્ટ) રન કરવા માટે વપરાય છે.

### 3. Create and kill processes (પ્રોસેસ બનાવવી અને બંધ કરવી) [1]

- આપણે કોડ દ્વારા નવી ચાઇલ્ડ પ્રોસેસ ક્રિએટ (Create) કરી શકીએ છીએ અને તેનું કામ પૂરું થઈ જાય અથવા તે અટવાઈ જાય, ત્યારે તેને કમાન્ડ આપીને ફોર્સફુલી કિલ (Kill - બંધ) પણ કરી શકીએ છીએ.
- **ઉદાહરણ:** `child.kill();` લખવાથી એ ચાઇલ્ડ પ્રોસેસ તરત જ બંધ થઈ જાય છે અને કમ્પ્યુટરની RAM ખાલી થઈ જાય છે.

અહીં તમારા સિલેબસના **તમામે તમામ ૧૦ મોડ્યુલ** એક પણ ટોપિક મિસ કર્યા વગર ખૂબ જ સાદી, સરળ અને વિગતવાર ગુજરાતી ભાષામાં પૂરા થાય છે.

હવે આપણે **Module 1: Introduction to Node.js**

થી શરૂઆત કરીને દરેક ટોપિકમાં એકદમ **Deep (ઊંડાણપૂર્વક)** જઈએ. આપણે દરેક કન્સેપ્ટની વ્યાખ્યા (Definition), સિન્ટેક્સ (Syntax), અંદરનું લોજિક અને રિયલ વર્લ્ડ ઉદાહરણો એક પણ મુદ્દો મિસ કર્યા વગર જોઈશું.

### Module 1: Introduction to Node.js (ઊંડાણપૂર્વક અભ્યાસ) [1]

#### ૧. What is Node.js (Node.js શું છે?) [1]

🔴 **વ્યાખ્યા (Definition):**

**Node.js** એ કોઈ પ્રોગ્રામિંગ ભાષા કે કોઈ પ્લગઇન નથી, પરંતુ તે કોમના **V8 JavaScript Engine** પર બનેલું એક ઓપન-સોર્સ, ક્રોસ-પ્લેટફોર્મ **JavaScript Runtime Environment** છે. તે બ્રાઉઝરની બહાર કમ્પ્યુટર કે સર્વર પર સીધો જાવાસ્ક્રિપ્ટ કોડ રન કરવાની સુવિધા આપે છે.

⚙️ **અંદરનું મેકેનિઝમ (How it works inside?):**

સામાન્ય રીતે જાવાસ્ક્રિપ્ટ બ્રાઉઝરની જેલ (Sandbox) માં બંધ હોય છે જેથી તે કમ્પ્યુટરની હાર્ડડિસ્ક કે નેટવર્કને એક્સેસ નથી કરી શકતી. Ryan Dahl નામના ડેવલપરે ૨૦૦૯ માં કોમ બ્રાઉઝરનું **V8 Engine** લીધું (જે C++ માં લખાયેલું છે) અને તેને કમ્પ્યુટરની ઓપરેટિંગ સિસ્ટમ સાથે સીધું કનેક્ટ કર્યું.

[તમારો JavaScript કોડ]

|



[Node.js C++ API] —► (હાર્ડડિસ્ક, નેટવર્ક, ફાઇલ્સ એક્સેસ)

|



[V8 Engine] —► [મશીન કોડ (૦ અને ૧)] —► [CPU પર રન થાય]

### સિન્ટેક્સ અને ફર્સ્ટ પ્રોગ્રામ (Syntax):

તમારા કમ્પ્યુટર પર જાવાસ્ક્રિપ્ટ રન કરવા માટે app.js નામની ફાઈલ બનાવો:

```
javascript
```

```
// સાદો અને સિમ્પલ જાવાસ્ક્રિપ્ટ કોડ
```

```
const message = "Node.js ની દુનિયામાં તમારું સ્વાગત છે!";
```

```
console.log(message);
```

Use code with caution.

**રન કરવાની રીત:** ટર્મિનલ/CMD ઓપન કરી લખો: node app.js

---

## ૨. Features of Node.js (Node.js ની મુખ્ય ખાસિયતો) [1]

જાવાસ્ક્રિપ્ટને બેકએન્ડ માટે આટલી શક્તિશાળી બનાવતી ૪ મુખ્ય ખાસિયતો આ પ્રમાણે છે:

### A. Asynchronous અને Non-blocking I/O [1]

- **I/O (Input/Output):** આનો મતલબ હાર્ડડિસ્ક પર ફાઈલ લખવી/વાંચવી કે નેટવર્ક પર ડેટા મોકલવો.
- Node.js માં બધી રિક્વેસ્ટ નોન-blocking હોય છે. જો ડેટાબેઝમાંથી મોટો ડેટા આવવામાં ૫ સેકન્ડ લાગવાની હોય, તો સર્વર એ ૫ સેકન્ડ સુધી બીજા યુઝર્સ માટે અટકી નથી જતું.

### B. Single-Threaded Event Loop Architecture [1]

- **Thread એટલે શું?:** એક થ્રેડ એટલે કમ્પ્યુટરના પ્રોસેસર દ્વારા એક સમયે ચાલતો એક સિંગલ લાઈન કોડ ફ્લો.
- Java કે PHP જેવા ફ્રેમવર્ક **Multi-Threaded** હોય છે. એટલે કે જેટલા નવા યુઝર્સ આવે, સર્વર તેટલા નવા થ્રેડ (RAM માં નવી જગ્યા) બનાવે છે. જો ૧૦,૦૦૦ યુઝર્સ એકસાથે આવે તો સર્વરની RAM ભરાઈ જાય અને તે કેશ થઈ જાય.
- Node.js **Single-Threaded** છે. તે બધા યુઝર્સને એક જ મુખ્ય થ્રેડ પર હેન્ડલ કરે છે, અને ભારે કામોને બેકએન્ડમાં ઓપરેટિંગ સિસ્ટમ કે Thread Pool ને સોંપી દે છે.

### C. Extremely Fast (ખૂબ જ ઝડપી) [1]

- ગૂગલનું V8 એન્જિન જાવાસ્ક્રિપ્ટ કોડને કમ્પાઈલ કરીને સીધો **Just-In-Time (JIT)** ટેકનોલોજીથી મશીન કોડમાં ફેરવે છે, તેથી તેનો એક્ઝિક્યુશન ટાઇમ મિલીસેકન્ડ્સમાં હોય છે.

### D. No Buffering (કોઈ બફરિંગ નહીં) [1]

- Node.js એપ્લિકેશન્સ ક્યારેય પણ ડેટાને આપ્પેઆપ્પો ભેગો (Buffer) કરીને આઉટપુટ નથી આપતી. તે ડેટાને નાના-નાના ટુકડાઓમાં (Chunks) સતત બહાર મોકલે છે.
-

### ૩. Concept (તેનો મુખ્ય કન્સેપ્ટ) [1]

Node.js નો કોર કન્સેપ્ટ છે: "તમામ જગ્યાએ એક જ ભાષા (JavaScript) વાપરો".

- **પહેલાની સ્થિતિ:** જો તમારે ફૂલ સ્ટેક ડેવલપર બનવું હોય, તો ફ્રન્ટએન્ડ માટે JavaScript અને બેકએન્ડ માટે PHP, Python કે Java શીખવી પડતી હતી.
- **Node.js નો કન્સેપ્ટ:** ફ્રન્ટએન્ડ (React/HTML/CSS) અને બેકએન્ડ (Node.js/Express) બંને એક જ પ્રોગ્રામિંગ લોજિક પર ચાલે છે. આનાથી કંપનીઓ અને ડેવલપર્સનો ઘણો સમય અને ખર્ચ બચે છે.

### ૪. Where to fit and not fit (ક્યાં વાપરવું અને ક્યાં ન વાપરવું?) [1]

સૌથી મોટો કન્ફ્યુઝન એ હોય છે કે કયા પ્રોજેક્ટમાં Node.js લેવું અને કયામાં ન લેવું. આને એક ડાયરેક્ટ સરખામણીથી સમજીએ:

**ક્યાં એકદમ બેસ્ટ ફિટ થાય છે (Where to Fit) [1]** **ક્યાં બિલકુલ ન વાપરવું (Where NOT to Fit) [1]**

**Real-time Applications:** ચેટ એપ્સ (WhatsApp ક્લોન), ઓનલાઇન ગેમ્સ.

**Heavy CPU Processing:** વીડિયો એડિટિંગ સોફ્ટવેર કે 3D રેન્ડરિંગ એપ્સ.

**Data Streaming:** લાઇવ વીડિયો કે ઓડિયો આપતી એપ્સ (Netflix, Spotify).

**Machine Learning:** મોટા ડેટા સેટ્સ પર મોડેલ ટ્રેનિંગ કરવા માટે (ત્યાં Python બેસ્ટ છે).

**I/O Bound Applications:** જ્યાં ડેટાબેઝમાંથી સતત ડેટા લાવવા-લખવાનો હોય.

**Cryptocurrency Mining:** જ્યાં પ્રોસેસરની ક્ષમતાનો સતત ૧૦૦% ઉપયોગ ગણતરીમાં થતો હોય.

**JSON APIs:** ફાસ્ટ અને લાઇટવેઇટ REST APIs બનાવવા માટે.

**સાદી મલ્ટી-પેજ બ્લોગ સાઇટ્સ:** જ્યાં ફક્ત સ્ટેટિક HTML જ બતાવવાનું હોય (ત્યાં PHP સહેલું પડે).

### ૫. Event-Driven programming style (ઇવેન્ટ-ડ્રિવન પ્રોગ્રામિંગ સ્ટાઇલ) [1]

✦ **સમજૂતી:**

ઇવેન્ટ-ડ્રિવનનો સાદો નિયમ છે: "કામ ત્યારે જ કરો જ્યારે કોઈ ઘટના બને".

Node.js સર્વર ચાલુ થઈને શાંતિથી બેસી રહે છે. જેવી કોઈ ઇવેન્ટ થાય (જેમ કે યુઝરે ફાઇલ અપલોડ કરી, કોઈ લિંક ઓપન કરી), એટલે Node.js તેના માટે નક્કી કરેલું ફંક્શન રન કરે છે.

🖥 **ઊંડાણપૂર્વકનું ઉદાહરણ:**

Node.js માં ઇવેન્ટ્સ કેવી રીતે રજીસ્ટર થાય છે અને ફાયર થાય છે તેનો સાદો કોડ:

```
javascript
```

```
// ઇવેન્ટ મોડ્યુલ લોડ કરો
```

```
const EventEmitter = require('events');
```

```
const myEmitter = new EventEmitter();
```

```
// ૧. ઇવેન્ટ લિસનર સેટ કરો (ઘટના બને ત્યારે શું કરવું?)  
myEmitter.on('orderPlaced', (orderId, foodName) => {  
  console.log(`ઇવેન્ટ મળી: ઓર્ડર નંબર ${orderId} (${foodName}) રસોડામાં મોકલી દીધો છે.`);  
});
```

```
// ૨. ઇવેન્ટ ફાયર કરો (ઘટના બની!)  
myEmitter.emit('orderPlaced', 101, 'ચીઝ બર્ગર');  
Use code with caution.
```

---

## ૬. What is Asynchronous (અસિન્ક્રોનસ એટલે શું?) [1]

આ કન્સેપ્ટને સમજવા માટે આપણે હોટેલનું ઉદાહરણ લઈએ:

- **Synchronous (બ્લોકિંગ):** એક વેઇટર ટેબલ-૧ નો ઓર્ડર લઈને રસોડામાં જાય છે. જ્યાં સુધી રસોડામાંથી ઓર્ડર બનીને ન આવે, ત્યાં સુધી વેઇટર રસોડાની બહાર જ ઉભો રહે છે (બ્લોક થઈ જાય છે). ટેબલ-૨ ના ગ્રાહકો ભૂખ્યા બેસી રહે છે.
- **Asynchronous (નોન-બ્લોકિંગ):** વેઇટર ટેબલ-૧ નો ઓર્ડર રસોડામાં આપીને તરત જ ટેબલ-૨ નો ઓર્ડર લેવા નીકળી જાય છે. જેવો ટેબલ-૧ નો ઓર્ડર રેડી થશે, રસોડામાંથી ઘંટડી વાગશે (Event ફાયર થશે) અને વેઇટર ઓર્ડર ટેબલ પર પહોંચાડી દેશે.

### કોડ દ્વારા તફાવત:

**Synchronous રીત (ખરાબ રીત - આખું સર્વર અટકી જશે):**

javascript

```
const fs = require('fs');
```

```
console.log("ફાઈલ વાંચવાનું શરૂ...");
```

```
// આ લાઈન જ્યાં સુધી પૂરી ન થાય ત્યાં સુધી નીચેનો કોડ રન નહિ થાય
```

```
const data = fs.readFileSync('large_data.txt');
```

```
console.log(data.toString());
```

```
console.log("આ લાઈન અટકેલી રહેશે!");
```

Use code with caution.

**Asynchronous રીત (Node.js ની સાચી રીત):**

javascript

```
const fs = require('fs');

console.log("ફાઇલ વાંચવાનું શરૂ...");

// ફાઇલ વાંચવાનું કામ બેકએન્ડમાં સોંપી દીધું
fs.readFile('large_data.txt', (err, data) => {
  if (err) throw err;
  console.log("ફાઇલનો ડેટા આવી ગયો!");
  console.log(data.toString());
});

console.log("આ લાઇન ફાઇલ વંચાય તે પહેલા જ પ્રિન્ટ થઈ જશે!");

Use code with caution.
```

---

અહીં **Module 1** ના તમામ ૬ ટોપિક એકદમ ડીપમાં પૂરા થાય છે.

આપણે આ જ રીતે **Module 2 (Installation & Runtime Setup)** માં ડીપમાં આગળ વધીએ? જો આ પ્રથમ મોડ્યુલના કોઈ કન્સેપ્ટમાં ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે **Module 2: Installation / Setup**

માં એકદમ ઊંડાણપૂર્વક (Deep) જઈએ. આ મોડ્યુલની અંદર Node.js નું આખું આર્કિટેક્ચર, રનટાઇમ કેવી રીતે કામ કરે છે, ઓપરેટિંગ સિસ્ટમ લેવલ પર તેનું સેટઅપ અને વેરિફિકેશન પ્રોસેસને વિગતવાર સમજીએ.

---

## Module 2: Installation / Setup (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Local environment setup (લોકલ એન્વાયરમેન્ટ સેટઅપ)

#### ✦ લોજિક અને આયોજન (The Setup Logic):

Node.js માં કામ કરવા માટે કમ્પ્યુટરને તૈયાર કરવું એટલે ફક્ત એક સોફ્ટવેર ઇન્સ્ટોલ કરવો એટલું જ નથી. આપણે પ્રોગ્રામિંગ કરવા માટે બે મુખ્ય કમ્પોનન્ટની જરૂર પડે છે:

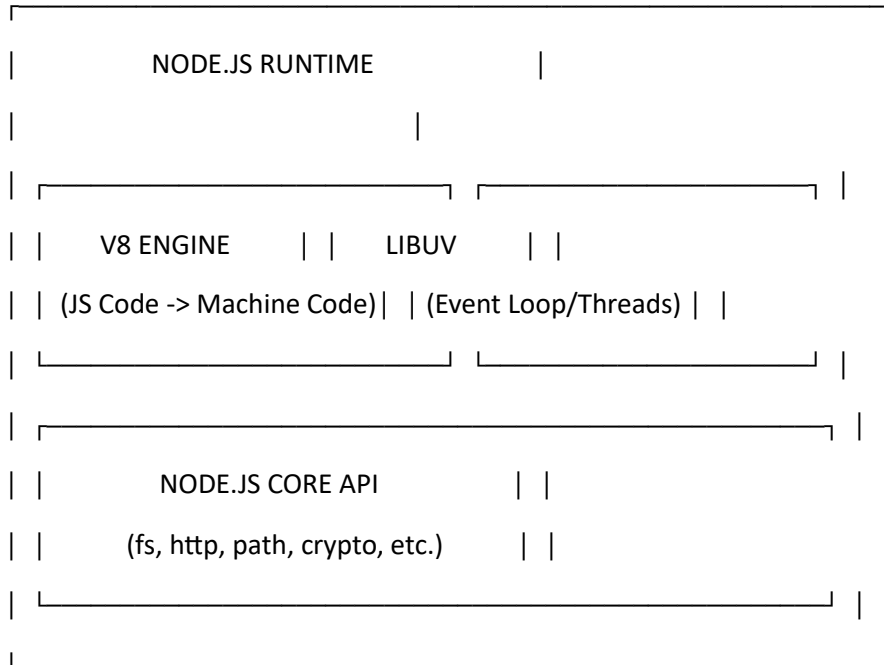
1. **Node.js Environment (The Machine):** આ આપણા જાવાસ્ક્રિપ્ટ કોડને રન કરવાનું કામ કરશે.
  2. **IDE / Text Editor (The Notepad for Developers):** કોડ લખવા અને ફાઇલ મેનેજ કરવા માટે **VS Code (Visual Studio Code)** ને બેસ્ટ ગણવામાં આવે છે. તેમાં લાઇવ ટર્મિનલ, ડીબગીંગ ટૂલ્સ અને ઇન્ટેલિસેન્સ (ઓટોમેટિક કોડ સજેશન) જેવી સુવિધાઓ મળે છે.
-

## ૨. Node.js runtime (Node.js રનટાઇમ)

### ⚙️ રનટાઇમ એટલે શું અને તે અંદરથી કેવો દેખાય છે?:

**Node.js Runtime** એ C++ માં લખાયેલું એક મોટું કન્ટેનર છે. આ કન્ટેનરની અંદર મુખ્ય ત્રણ વસ્તુઓ હોય છે:

- **V8 Engine:** આ ગૂગલનું એન્જિન છે જે જાવાસ્ક્રિપ્ટ કોડ (Human Readable Code) ને કમ્પ્યુટર સમજી શકે તેવા મશીન કોડ (Machine Code) માં ફેરવે છે.
- **libuv (C++ Library):** આ Node.js નો સૌથી મહત્વનો ભાગ છે. તે ઓપરેટિંગ સિસ્ટમના થ્રેડ પૂલ (Thread Pool), ઇવેન્ટ લૂપ (Event Loop), ફાઇલ સિસ્ટમ અને નેટવર્કિંગના કાર્યોને અસિન્ક્રોનસ રીતે હેન્ડલ કરે છે.
- **Node.js Core API:** ઇન-બિલ્ટ મોડ્યુલ્સ જેવા કે fs, path, http, crypto જે આપણને જાવાસ્ક્રિપ્ટ દ્વારા સીધા કમ્પ્યુટરના હાર્ડવેર સાથે વાત કરવાની મંજૂરી આપે છે.



## ૩. Download source code (સોર્સ કોડ ડાઉનલોડ કરવો)

### 📦 સાચી પસંદગી (LTS vs Current):

જ્યારે તમે ઓફિશિયલ વેબસાઇટ **nodejs.org** પર જાઓ છો, ત્યારે તમને ડાઉનલોડ કરવા માટે બે ઓપ્શન મળે છે:

- **LTS (Long Term Support):**
  - **લાભ:** આ વર્ઝન એકદમ સ્ટેબલ, બગ-ફ્રી અને ટેસ્ટેડ હોય છે. મોટી કંપનીઓ અને પ્રોડક્શન એપ્લિકેશન્સમાં હંમેશા આ જ વર્ઝન વપરાય છે. આને લાંબા સમય સુધી સિક્યોરિટી અપડેટ્સ મળતા રહે છે.
- **Current:**

- **લાભ:** આમાં લેટેસ્ટ ફીચર્સ અને નવી ટેકનોલોજી ઉમેરેલી હોય છે. પરંતુ આમાં બગ્સ (ભૂલો) હોવાની શક્યતા વધારે હોય છે, તેથી આ ફક્ત ટેસ્ટિંગ અને પ્રયોગો કરવા માટે જ યોગ્ય છે.

---

## ૪. Installation on OS (ઓપરેટિંગ સિસ્ટમ પર ઇન્સ્ટોલેશન)

દરેક ઓપરેટિંગ સિસ્ટમ પર સેટઅપ કરવાની રીત અલગ-અલગ હોય છે:

### A. Windows ઓપરેટિંગ સિસ્ટમ પર:

- વેબસાઇટ પરથી સાદી .msi (Windows Installer) ફાઇલ ડાઉનલોડ કરો.
- ફાઇલ ઓપન કરીને "Next" પર ક્લિક કરો, લાઇસન્સ એગ્રીમેન્ટ સ્વીકારો.
- ખૂબ જ મહત્વનો સ્ટેપ:** ઇન્સ્ટોલેશન વખતે "Add to PATH" નો ઓપ્શન ઓટોમેટિક સિલેક્ટ થયેલો જ હોવો જોઈએ. આનાથી કમ્પ્યુટરના પર્યાવરણ વેરીએબલ્સ (Environment Variables) સેટ થાય છે અને તમે કમ્પ્યુટરના કોઈ પણ ખૂણેથી કે ટર્મિનલમાંથી node કમાન્ડ રન કરી શકો છો.

### B. macOS (Mac) પર:

- વેબસાઇટ પરથી .pkg ફાઇલ ડાઉનલોડ કરીને ડબલ ક્લિકથી ઇન્સ્ટોલ કરી શકાય છે.
- અથવા ડેવલપર્સની પ્રખ્યાત રીત **Homebrew** ટર્મિનલ કમાન્ડથી પણ ઇન્સ્ટોલ કરી શકાય છે: `brew install node`

### C. Linux (Ubuntu / Debian) પર:

લિનક્સમાં ડાયરેક્ટ પેકેજ મેનેજર અથવા **NodeSource** રિપોઝિટરીનો ઉપયોગ થાય છે. ટર્મિનલમાં નીચેના કમાન્ડ રન કરવા પડે છે:

```
bash
```

```
# NodeSource પરથી વર્ઝન સેટઅપ કરવું
```

```
curl -fsSL https://nodesource.com | sudo -E bash -
```

```
# Node.js ઇન્સ્ટોલ કરવું
```

```
sudo apt-get install -y nodejs
```

Use code with caution.

---

## ૫. Verify (વેરિફિકેશન - ચેક કરવું)

ઇન્સ્ટોલેશન પૂરું થયા પછી સિસ્ટમ બરાબર સેટ થઈ છે કે નહીં તે ચેક કરવું જરૂરી છે.

### ટર્મિનલ ટેસ્ટિંગ (Commands):

તમારું Command Prompt (Windows) અથવા Terminal (Mac/Linux) ઓપન કરો અને નીચેના બે અલગ-અલગ કમાન્ડ લખીને એન્ટર મારો:

```
bash
```

# ૧. Node.js નું વર્ઝન ચેક કરવા

node -v

# આઉટપુટ આના જેવું દેખાશે: v20.11.0 (અથવા તમારું લેટેસ્ટ વર્ઝન)

# ૨. NPM (Node Package Manager) નું વર્ઝન ચેક કરવા

npm -v

# આઉટપુટ આના જેવું દેખાશે: 10.2.4

Use code with caution.

### **REPL (Read-Eval-Print Loop) ટેસ્ટિંગ:**

Node.js માં ફાઇલ બનાવ્યા વગર ડાયરેક્ટ કમાન્ડ લાઇન પર નાનો જાવાસ્ક્રિપ્ટ કોડ ટેસ્ટ કરવા માટે **REPL** નો ઉપયોગ થાય છે.

1. ટર્મિનલમાં ફક્ત node લખીને એન્ટર મારો.
2. તમારું ટર્મિનલ બદલાઈ જશે અને ત્યાં તમે જાવાસ્ક્રિપ્ટ રન કરી શકશો.
3. ઉદાહરણ તરીકે લખો: 10 + 20 અને એન્ટર મારો, તરત જ નીચે જવાબ મળશે 30.
4. REPL માંથી બહાર નીકળવા માટે Ctrl + C બે વાર દબાવો અથવા .exit લખો.

---

અહીં **Module 2** ના તમામ ૫ ટોપિક એકદમ ડીપમાં પૂરા થાય છે.

આપણે આ જ સીક્વન્સમાં **Module 3: Node Package Manager (NPM)** માં ડીપમાં આગળ વધીએ? જો આ મોડ્યુલ સેટઅપમાં કોઈ પ્રશ્ન હોય તો પૂછી શકો છો!

ચાલો, હવે આપણે **Module 3: Node Package Manager (NPM)**

માં એકદમ ઊંડાણપૂર્વક (Deep) જઈએ. આ મોડ્યુલની અંદર પેકેજ કેવી રીતે મેનેજ થાય છે, લોકલ અને ગ્લોબલ સેટઅપ વચ્ચે શું તફાવત છે, અને package.json ફાઇલ અંદરથી કેવી રીતે કામ કરે છે તે બધું વિગતવાર સમજીએ.

---

## **Module 3: Node Package Manager - NPM (ઊંડાણપૂર્વક અભ્યાસ)**

### **૧. Install module by NPM (NPM દ્વારા મોડ્યુલ ઇન્સ્ટોલ કરવું)**

#### **NPM શું છે?**

**NPM** એ દુનિયાની સૌથી મોટી સોફ્ટવેર રજિસ્ટ્રી છે. તે Node.js ઇન્સ્ટોલ કરતી વખતે તેની સાથે જ ઓટોમેટિક ઇન્સ્ટોલ થઈ જાય છે. દુનિયાભરના ડેવલપર્સ બનાવેલા તૈયાર કોડના ટુકડા (જેને પેકેજ અથવા મોડ્યુલ કહેવાય છે) આપણે આપણી એપ્લિકેશનમાં વાપરવા માટે NPM નો ઉપયોગ કરીએ છીએ.

#### **પ્રોજેક્ટની શરૂઆત અને ઇન્સ્ટોલેશન (Initialization):**

કોઈપણ પ્રોજેક્ટમાં પેકેજ ઇન્સ્ટોલ કરતા પહેલા પ્રોજેક્ટને ઇનિશિયલાઇઝ (Initialize) કરવો પડે છે.

- **કમાન્ડ:** npm init (આનાથી પ્રોજેક્ટના નામ અને વર્ઝન વિશે સવાલો પૂછાશે).
- **શોર્ટકટ કમાન્ડ:** npm init -y (બધા સવાલોના જવાબ આપોઆપ "Yes" થઈ જશે અને પ્રોજેક્ટ સેટ થઈ જશે).

આ કમાન્ડ રન કરવાથી તમારા ફોલ્ડરમાં **package.json** નામની એક મુખ્ય ફાઇલ બનશે. આ ફાઇલ તમારા પ્રોજેક્ટનું હૃદય છે, જે બધા જ ઇન્સ્ટોલ કરેલા પેકેજોનું લિસ્ટ સાચવી રાખે છે.

### પેકેજ ઇન્સ્ટોલ કરવાનો કમાન્ડ:

જ્યારે આપણે કોઈ પેકેજ ઇન્સ્ટોલ કરીએ, જેમ કે ડેટા ફોર્મેટિંગ માટે lodash પેકેજ:

```
bash
```

```
npm install lodash
```

Use code with caution.

આ કમાન્ડ રન થવાથી બે વસ્તુઓ થશે:

1. તમારા ફોલ્ડરમાં **node\_modules** નામનું મોટું ફોલ્ડર બનશે, જેમાં ડાઉનલોડ થયેલો અસલી કોડ સેવ થશે.
2. package.json ની અંદર "dependencies" નામના ઓબ્જેક્ટમાં એ પેકેજનું નામ અને વર્ઝન ઉમેરાઈ જશે.

## ૨. Global vs Local setup (ગ્લોબલ વર્સિસ લોકલ સેટઅપ)

પેકેજને ઇન્સ્ટોલ કરવાની બે મુખ્ય પદ્ધતિઓ છે, જેને સમજવી ખૂબ જ જરૂરી છે:

| લક્ષણ          | લોકલ સેટઅપ (Local Installation)                               | ગ્લોબલ સેટઅપ (Global Installation)                               |
|----------------|---------------------------------------------------------------|------------------------------------------------------------------|
| કમાન્ડ         | npm install package_name                                      | npm install -g package_name                                      |
| ક્યાં સેવ થાય? | ફક્ત તમારા ચાલુ પ્રોજેક્ટના ફોલ્ડરમાં (node_modules ની અંદર). | તમારા આખા કમ્પ્યુટરના મુખ્ય સિસ્ટમ ડિરેક્ટરીમાં.                 |
| ઉપયોગ          | ફક્ત એ જ પ્રોજેક્ટની અંદર require() કે import કરી શકાય.       | ટર્મિનલ/CMD માંથી કમાન્ડ લાઇન ટૂલ તરીકે ગમે ત્યાંથી રન કરી શકાય. |
| ઉદાહરણ         | express, mongoose, lodash (પ્રોજેક્ટ લોજિક પેકેજ).            | nodemon (સર્વર ઓટો-રીસ્ટાર્ટ કરવા), rimraf (ફોલ્ડર ડીલીટ કરવા).  |

## ૩. Update Module (મોડ્યુલ અપડેટ કરવું)

સમય જતાં ઓપન-સોર્સ પેકેજોમાં બગ્સ ફિક્સ થાય છે અને નવા ફીચર્સ આવે છે, તેથી તેને અપડેટ કરવા પડે છે.

### ⚙️ અપડેટ પ્રોસેસ (Commands):

- **આઉટડેટેડ પેકેજ ચેક કરવા:** npm outdated (આ કમાન્ડથી ખબર પડશે કે કયા પેકેજનું નવું વર્ઝન માર્કેટમાં આવ્યું છે).
- **ચોક્કસ પેકેજ અપડેટ કરવા:** npm update package\_name
- **બધા જ પેકેજ એકસાથે અપડેટ કરવા:** npm update

### 🧠 સેમવેર (Semantic Versioning - ^ અને ~ નો જાદુ):

package.json માં વર્ઝન નંબરની આગળ બે ખાસ ચિહ્નો જોવા મળે છે, જેમ કે "lodash": "^4.17.21":

- **^ (Caret - કેરેટ):** આનો મતલબ એવો થાય કે જ્યારે તમે અપડેટ કમાન્ડ ચલાવશો, ત્યારે તે માત્ર માઇનર અપડેટ્સ (Minor / Bug fixes) જ લેશે (જેમ કે 4.18.0 સુધી જશે), પણ મેઇન વર્ઝન બદલશે નહીં (જેથી તમારો કોડ તૂટે નહીં).
- **~ (Tilde - ટિલ્ડ):** આ ફક્ત છેલ્લો ડિજિટ એટલે કે પેચ વર્ઝન (Patch fixes) જ અપડેટ થવા દેશે (જેમ કે 4.17.30 સુધી જ જશે).

---

### ૪. CRUD Module (મોડ્યુલનું મેનેજમેન્ટ)

ડેટાબેઝની જેમ જ આપણે NPM પેકેજીસ સાથે પણ CRUD (Create, Read, Update, Delete) ઓપરેશન્સ કરી શકીએ છીએ:

- **Create (પેકેજ પ્રોજેક્ટમાં લાવવું):**  
npm install package\_name કરવાથી પેકેજ ઇન્સ્ટોલ થાય છે.
- **Read (ઇન્સ્ટોલ થયેલા પેકેજ જોવા):**  
npm list કમાન્ડ ટર્મિનલમાં લખવાથી તમારા પ્રોજેક્ટમાં કયા-કયા પેકેજ કયા વર્ઝન સાથે ઇન્સ્ટોલ છે તેનું આખું લિસ્ટ (Tree structure) સ્ક્રીન પર દેખાશે. જો ફક્ત મેઇન પેકેજ જોવા હોય તો npm list --depth=0 લખી શકાય.
- **Update (વર્ઝન બદલવું):**  
npm update package\_name કમાન્ડથી પેકેજ લેટેસ્ટ સ્ટેબલ વર્ઝન પર શિફ્ટ થાય છે.
- **Delete (પેકેજ કાયમ માટે હટાવવું):**  
જો પ્રોજેક્ટમાં કોઈ પેકેજ નકામું પડ્યું હોય, તો તેને રાખવાથી પ્રોજેક્ટની સાઇઝ મોટી થાય છે. તેને હટાવવા માટે:

bash

npm uninstall package\_name

Use code with caution.

આ કમાન્ડ node\_modules માંથી એ પેકેજનો કોડ ડીલીટ કરી નાખશે અને package.json માંથી તેનું નામ પણ સાફ કરી દેશે.

---

અહીં **Module 3** ના તમામ ટોપિક એકદમ ડીપમાં પૂરા થાય છે.

આપણે આ જ રીતે **Module 4 (Express Framework)** માં ઊંડાણપૂર્વક આગળ વધીએ? જો આ મોડ્યુલ (NPM) માં કોઈ પ્રશ્ન હોય તો પૂછી શકો છો!

Try without personalization

ચાલો, હવે આપણે

**Module 4: Express Framework** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ વેબ ડેવલપમેન્ટ માટે સૌથી મહત્વનું છે. આપણે દરેક પોઇન્ટની વ્યાખ્યા, તેનું આંતરિક લોજિક અને પ્રેક્ટિકલ કોડ ઉદાહરણો સાથે સમજીશું.

---

## Module 4: Express Framework (ઊંડાણપૂર્વક અભ્યાસ) [1]

### ૧. What is Express (Express શું છે?) [1]

#### ❖ વ્યાખ્યા (Definition):

**Express.js** એ Node.js માટેનું સૌથી લોકપ્રિય, ફાસ્ટ, અન-ઓપિનિઓનેટેડ (Unopinionated) અને મિનિમલિસ્ટ વેબ એપ્લિકેશન ફ્રેમવર્ક છે.

- **અન-ઓપિનિઓનેટેડ એટલે શું?**: આ ફ્રેમવર્ક ડેવલપર પર કોઈ કડક નિયમો નથી થોપતું. તમે તમારા પ્રોજેક્ટનું ફોલ્ડર સ્ટ્રક્ચર અને લોજિક ગમે તે રીતે ગોઠવવા માટે સંપૂર્ણ સ્વતંત્ર છો.
- **તે કેમ વપરાય છે?**: Node.js ના પોતાના http મોડ્યુલથી સર્વર બનાવતી વખતે યુઝરના ડેટા સોર્ટ કરવા, રાઉટીંગ હેન્ડલ કરવા કે મિડલવેર સેટ કરવા માટે સેંકડો લાઇનનો મેન્યુઅલ કોડ લખવો પડે છે. Express તે બધા જ કામોને માત્ર થોડીક લાઇનોના સાદા ફંક્શન્સમાં ફેરવી દે છે.

---

### ૨. Setup Express (Express સેટઅપ કરવું) [1]

પ્રોજેક્ટ શરૂ કરવા માટેના સ્ટેપ્સ અને બેઝિક સર્વર સેટઅપ:

bash

# ૧. નવું ફોલ્ડર બનાવી તેમાં પ્રોજેક્ટ શરૂ કરો

npm init -y

# ૨. એક્સપ્રેસ મોડ્યુલ ઇન્સ્ટોલ કરો

npm install express

Use code with caution.

#### 📁 સર્વર કોડ (app.js):

javascript

```
const express = require('express');
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// સર્વરને પોર્ટ 3000 પર લાઈવ (ચાલુ) કરવું
```

```
app.listen(PORT, () => {
```

```
  console.log(`સર્વર સફળતાપૂર્વક http://localhost:${PORT} પર ચાલુ થઈ ગયું છે.`);
```

```
});
```

Use code with caution.

---

### ૩. Request And Response (રિક્વેસ્ટ અને રિસ્પોન્સ) [1]

જ્યારે ક્લાયન્ટ અને સર્વર વચ્ચે વાતચીત થાય છે, ત્યારે એક્સપ્રેસ તેને બે મુખ્ય ઓબ્જેક્ટ્સ દ્વારા હેન્ડલ કરે છે:

- **Request Object (req):** બ્રાઉઝર કે ક્લાયન્ટ તરફથી આવતી તમામ વિગતો આમાં હોય છે. જેમ કે યુઝર કયું બ્રાઉઝર વાપરે છે, URL માં કયો ડેટા મોકલ્યો છે, કે ફોર્મનો ડેટા શું છે.
- **Response Object (res):** સર્વર પ્રક્રિયા પૂરી કરીને જે વળતો જવાબ મોકલે છે તે આ ઓબ્જેક્ટ દ્વારા નક્કી થાય છે.

#### ડેટા મોકલવાની અલગ-અલગ રીતો:

javascript

```
app.get('/test', (req, res) => {
```

```
  // res.send(): સીધું લખાણ અથવા HTML મોકલવા માટે
```

```
  // res.send("<h1>હેલો</h1>");
```

```
  // res.json(): API બનાવવા માટે JSON ડેટા મોકલવા (સૌથી વધુ વપરાય છે)
```

```
  res.json({ message: "સફળતાપૂર્વક કનેક્ટ થયું", status: true });
```

```
});
```

Use code with caution.

---

### ૪. Handling Routes (રાઉટ્સ હેન્ડલ કરવા) [1]

#### રાઉટીંગ એટલે શું?

યુઝર બ્રાઉઝરની યુઆરએલ (URL) બારમાં જે એડ્રેસ લખે, તે પ્રમાણે તેને અલગ-અલગ પેજ કે ડેટા બતાવવાની પ્રક્રિયાને **Routing** કહેવાય છે.

#### રાઉટીંગનું ઉદાહરણ:

javascript

```
// હોમ પેજ રાઉટ
app.get('/', (req, res) => {
  res.send("અહીં હોમ પેજ (Main Page) છે.");
});

// પ્રોફાઇલ પેજ રાઉટ
app.get('/profile', (req, res) => {
  res.send("અહીં યુઝર પ્રોફાઇલ પેજ છે.");
});

// જો કોઈ ખોટો પાથ નાખે (404 પેજ)
app.use((req, res) => {
  res.status(404).send("માફ કરશો, આ પેજ ઉપલબ્ધ નથી.");
});

Use code with caution.
```

---

#### ૫. Route Middleware (રાઉટ મિડલવેર) [1]

##### ✦ ઊંડાણપૂર્વક કન્સેપ્ટ:

મિડલવેર એટલે એક એવું ચોકીદાર ફંક્શન, જે **Request** આવે અને **Response** જાય તેની વચ્ચે રન થાય છે. તેને 3 પેરામીટર્સ મળે છે: req, res, અને next. જો મિડલવેર સંતોષકારક હોય તો તે next() ફંક્શન બોલાવીને રિક્વેસ્ટને આગળના રાઉટ લોજિક તરફ મોકલે છે.

##### 📄 કસ્ટમ મિડલવેરનું ઉદાહરણ:

```
javascript

// ૧. લોગ રાખવા માટેનું એક સાદું મિડલવેર ફંક્શન
const myLogger = (req, res, next) => {
  console.log(`રિક્વેસ્ટ આવી: ${req.method} URL: ${req.url}`);
  next(); // આના વગર રિક્વેસ્ટ આગળ નહીં વધે અને ગોળ-ગોળ ફરશે
};

// ૨. મિડલવેરને આખી એપ્લિકેશનમાં ચાલુ કરવું
app.use(myLogger);
```

Use code with caution.

---

## ૬. Objects (ઓબ્જેક્ટ્સ) [1]

યુઝર પાસેથી આવતો ડેટા વાંચવા માટે એક્સપ્રેસમાં નીચેના ઓબ્જેક્ટ પ્રોપર્ટીઝ વપરાય છે:

### A. req.query (URL Query Parameters)

જ્યારે URL ની પાછળ ? કરીને ડેટા મોકલાય, જેમ કે: `http://localhost:3000/search?city=ahmedabad`

javascript

```
app.get('/search', (req, res) => {  
  const city = req.query.city; // 'ahmedabad' મળશે  
  res.send(`તમે શોધેલું શહેર: ${city}`);  
});
```

Use code with caution.

### B. req.params (URL Path Parameters)

જ્યારે URL ની અંદર જ ડાયનેમિક આઈડી પાસ કરવાનો હોય, જેમ કે: `/user/101`

javascript

```
app.get('/user/:id', (req, res) => {  
  const userId = req.params.id; // '101' મળશે  
  res.send(`યુઝર આઈડી: ${userId}`);  
});
```

Use code with caution.

### C. req.body (Form / JSON Data)

જ્યારે POST રિક્વેસ્ટ દ્વારા સિક્રેટ ફોર્મ ડેટા મોકલવામાં આવે. આ ડેટા વાંચવા માટે એક્સપ્રેસમાં એક ઇન-બિલ્ટ મિડલવેર લાઇન ઉમેરવી પડે છે:

javascript

```
app.use(express.json()); // JSON ડેટા વાંચવા માટે જરૂરી મિડલવેર
```

```
app.post('/register', (req, res) => {  
  const username = req.body.name;  
  res.send(`રજીસ્ટ્રેશન સફળ: ${username}`);  
});
```

Use code with caution.

---

### ૭. Cookies Management (કુકીઝ મેનેજમેન્ટ) [1]

યુઝરના બ્રાઉઝરમાં નાની કી-વેલ્યુ માહિતી સેવ કરવા માટે કુકીઝ વપરાય છે. આના માટે cookie-parser નામનું પેકેજ ઇન્સ્ટોલ કરવું પડે છે.

bash

```
npm install cookie-parser
```

Use code with caution.

#### કુકીઝ કોડ:

javascript

```
const cookieParser = require('cookie-parser');
```

```
app.use(cookieParser());
```

```
// કુકી સેટ કરવી
```

```
app.get('/set-cookie', (req, res) => {  
  res.cookie('userToken', 'abc123xyz', { maxAge: 900000, httpOnly: true });  
  res.send('કુકી બ્રાઉઝરમાં સેવ થઈ ગઈ છે.');
```

```
});
```

```
// કુકી વાંચવી
```

```
app.get('/get-cookie', (req, res) => {  
  const token = req.cookies.userToken;  
  res.send(`સેવ થયેલી કુકી ટોકન: ${token}`);  
});
```

Use code with caution.

---

### ૮. File upload (ફાઇલ અપલોડ) [1]

યુઝર પાસેથી પ્રોફાઇલ ઇમેજ કે ફાઇલ સર્વર પર અપલોડ કરાવવા માટે **multer** નામનું મિડલવેર પેકેજ બેસ્ટ છે.

bash

```
npm install multer
```

Use code with caution.

### ફાઇલ અપલોડ કોડ:

javascript

```
const multer = require('multer');

// અપલોડ થયેલી ફાઇલો 'uploads/' નામના ફોલ્ડરમાં જશે
const upload = multer({ dest: 'uploads/' });

// 'avatar' એ તમારા HTML ફોર્મ ઇનપુટનું નામ હોવું જોઈએ
app.post('/upload-profile', upload.single('avatar'), (req, res) => {
  console.log(req.file); // ફાઇલની બધી જ ડિટેલ્સ અહીં દેખાશે
  res.send('ફાઇલ સફળતાપૂર્વક અપલોડ થઈ ગઈ છે.');
```

Use code with caution.

---

## ૯. HTTP methods (HTTP મેથડ્સ) [1]

ડેટાબેઝ સાથેની CRUD પ્રક્રિયાઓને વ્યવસ્થિત રુટ આપવા માટે અલગ-અલગ મેથડ્સ વપરાય છે:

javascript

```
// ૧. GET: ડેટા રીડ (વાંચવા) કરવા માટે
app.get('/api/books', (req, res) => {
  res.json({ message: "બધી બુક્સનું લિસ્ટ" });
});

// ૨. POST: નવો ડેટા ક્રિએટ (બનાવવા) કરવા માટે
app.post('/api/books', (req, res) => {
  res.json({ message: "નવી બુક ઉમેરાઈ ગઈ" });
});

// ૩. PUT: જૂના ડેટાને સંપૂર્ણ અપડેટ (બદલવા) કરવા માટે
app.put('/api/books/:id', (req, res) => {
  res.json({ message: `_${req.params.id}_ નંબરની બુક અપડેટ થઈ` });
});
```

```
});
```

```
// ૪. DELETE: ડેટા ડીલીટ (કાઢી નાખવા) કરવા માટે
```

```
app.delete('/api/books/:id', (req, res) => {
```

```
  res.json({ message: `${req.params.id} નંબરની બુક ડીલીટ થઈ` });
```

```
});
```

Use code with caution.

---

અહીં **Module 4** ના તમામે તમામ ૯ પોઇન્ટ્સ એકદમ ઊંડાણપૂર્વક અને પ્રેક્ટિકલ કોડ સાથે પૂરા થાય છે.

હવે આપણે આગળ વધીને **Module 5: Buffers and Streams** માં ઊંડાણપૂર્વક જઈએ? જો આ મોડ્યુલ (Express) ની કોઈ કોડ કે કન્સેપ્ટમાં મૂઝવણ હોય તો કહી શકો છો!

ચાલો, હવે આપણે

**Module 5: Buffers and Streams** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ Node.js ની હાઇ-પર્ફોર્મન્સ મેમરી મેનેજમેન્ટ સિસ્ટમ સમજવા માટે ખૂબ જ જરૂરી છે.

---

## Module 5: Buffers and Streams (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. What is buffers and Streams (બફર્સ અને સ્ટ્રીમ્સ શું છે?)

#### ✦ Buffer (બફર):

**Buffer** એ કમ્પ્યુટરની રેમ (RAM) ની અંદર બનેલી એક નાની, ફિક્સ સાઈઝની જગ્યા છે, જે કાચા બાયનરી ડેટા (Binary Data - 0 અને 1) ને ટેમ્પરરી સાચવવા માટે વપરાય છે.

- જાવાસ્ક્રિપ્ટ સામાન્ય રીતે ફક્ત સ્ટ્રિંગ કે ઓબ્જેક્ટ સમજે છે. પરંતુ જ્યારે ફાઇલ રીડ કરવી હોય કે નેટવર્ક પર ઓડિયો મોકલવો હોય, ત્યારે ડેટા બાયનરી ફોર્મેટમાં હોય છે. Node.js માં આ બાયનરી ડેટાને હેન્ડલ કરવા માટે ગ્લોબલ **Buffer** ક્લાસ આપેલો છે.

#### ✦ Stream (સ્ટ્રીમ):

**Stream** એ મોટા ડેટાને એક જગ્યાએથી બીજી જગ્યાએ ટુકડે-ટુકડે (Chunks માં) સતત મોકલવાની પ્રક્રિયા છે.

- આને નદીના વહેતા પાણી સાથે સરખાવી શકાય. ડેટા આખેઆખો એકસાથે લોડ થવાને બદલે નાના-નાના ભાગમાં વહેતો રહે છે.

---

### ૨. Benefit of Streams over buffers (બફર્સ કરતાં સ્ટ્રીમ્સના ફાયદા)

આ બંને વચ્ચેનો તફાવત સમજવો એ એડવાન્સ ડેવલપમેન્ટ માટે ખૂબ જ જરૂરી છે:

#### 📺 રિયલ-વર્લ્ડ ઉદાહરણ (વીડિયો પ્લેયર):

- **Buffer રીત:** જો તમે 1 GB ની મુવી ઓનલાઇન જોતા હોવ અને સર્વર બફર વાપરતું હોય, તો જ્યાં સુધી આખેઆખી 1 GB મુવી બેકગ્રાઉન્ડમાં ડાઉનલોડ નહીં થાય, ત્યાં સુધી પ્લેયર ચાલુ જ નહીં થાય. આનાથી યુઝરની RAM ભરાઈ જશે.
- **Stream રીત:** મુવીના નાના-નાના ટુકડા (Chunks) બ્રાઉઝર પર સતત આવતા રહે છે અને વીડિયો તરત જ ચાલુ થઈ જાય છે. YouTube અને Netflix આ જ રીતે કામ કરે છે.

#### મુખ્ય ફાયદા:

1. **મેમરીની બચત (Memory Efficiency):** 1 GB ની ફાઈલ પ્રોસેસ કરવા માટે પણ માત્ર થોડાક KB ની RAM જ વપરાય છે.
2. **સમયની બચત (Time Efficiency):** આખો ડેટા લોડ થાય તેની રાહ જોવાની જરૂર નથી, પહેલો ટુકડો મળતા જ પ્રોસેસિંગ શરૂ થઈ જાય છે.

---

### 3. Create / Write / Read operation on Buffers (બફર્સ પર ઓપરેશન્સ)

Node.js માં બફર સાથે કામ કરવા માટે નીચે મુજબની સ્ટાન્ડર્ડ સિન્ટેક્સ વપરાય છે:

#### બફર બનાવવું (Create):

javascript

```
// 10 બાઈટ્સનું ખાલી બફર બનાવો
```

```
const buf1 = Buffer.alloc(10);
```

```
// કોઈ સ્ટ્રિંગ ડેટા પરથી ડાયરેક્ટ બફર બનાવવું
```

```
const buf2 = Buffer.from("Hello");
```

```
console.log(buf2); // આઉટપુટ હેક્સાડેસિમલ બાયનરી કોડમાં દેખાશે: <Buffer 48 65 6c 6c 6f>
```

Use code with caution.

#### બફરમાં લખવું અને વાંચવું (Write & Read):

javascript

```
const myBuf = Buffer.alloc(12);
```

```
// બફરમાં લખાણ લખવું
```

```
myBuf.write("NodeJS");
```

```
// બફરના બાયનરી ડેટાને પાછો વાંચવો
```

```
console.log(myBuf.toString()); // આઉટપુટ: NodeJS
```

Use code with caution.

---

## ૪. Process on Buffers (બફર્સ પર પ્રોસેસ)

બફર્સ ડેટા સાથે એડવાન્સ કામ કરવા માટે નીચેની મેથડ્સ વપરાય છે:

### A. બફર્સને જોડવા (Concatenation):

javascript

```
const b1 = Buffer.from("ગુજરાત ");  
const b2 = Buffer.from("ટૂંકાંમ");  
const totalBuf = Buffer.concat([b1, b2]);
```

```
console.log(totalBuf.toString()); // આઉટપુટ: ગુજરાત ટૂંકાંમ
```

Use code with caution.

### B. બફર કોપી કરવું (Copy):

javascript

```
const source = Buffer.from("ABC");  
const target = Buffer.alloc(3);
```

```
source.copy(target);
```

```
console.log(target.toString()); // આઉટપુટ: ABC
```

Use code with caution.

---

## ૫. Read / Write data by Streams (સ્ટ્રીમ્સ દ્વારા ડેટા રીડ/રાઇટ કરવો)

Node.js ની કોરે **fs (File System)** મોડ્યુલની મદદથી આપણે વાઈવ સ્ટ્રીમ બનાવી શકીએ છીએ. સ્ટ્રીમ્સ મુખ્ય બે પ્રકારની હોય છે: Readable અને Writable.



### ફાઈલ વાંચવા માટે (Readable Stream):

javascript

```
const fs = require('fs');
```

```
// મોટી ફાઈલ વાંચવા માટેની સ્ટ્રીમ
```

```
const readStream = fs.createReadStream('large_data.txt', { encoding: 'utf8' });
```

// જેવો ડેટાનો એક નાનો ટુકડો (Chunk) મળશે એટલે આ ઇવેન્ટ ચાલુ થશે

```
readStream.on('data', (chunk) => {  
  console.log("--- નવો ટુકડો મળ્યો ---");  
  console.log(chunk);  
});
```

// ફાઇલ આખી વંચાઈ જાય ત્યારે

```
readStream.on('end', () => {  
  console.log("ફાઇલ વાંચવાનું કામ પૂરું થયું.");  
});
```

Use code with caution.

#### ફાઇલમાં લખવા માટે (Writable Stream):

javascript

```
const fs = require('fs');
```

```
const writeStream = fs.createWriteStream('new_output.txt');
```

// સ્ટ્રીમની મદદથી ટુકડે-ટુકડે ડેટા ફાઇલમાં લખવો

```
writeStream.write("પહેલી લાઇન લખી.\n");
```

```
writeStream.write("બીજી લાઇન લખી.\n");
```

// સ્ટ્રીમ બંધ કરવી

```
writeStream.end();
```

Use code with caution.

---

#### ૬. Pipeline (પાઇપલાઇન)

##### કન્સેપ્ટ:

જ્યારે આપણે કોઈ રીડ સ્ટ્રીમ માંથી ડેટા લઈને તેને રાઇટ સ્ટ્રીમમાં મોકલવો હોય, ત્યારે જો વચ્ચે કોઈ નેટવર્ક કે ફાઇલ એરર આવે તો સાદી પદ્ધતિમાં સ્ટ્રીમ્સ ખુલ્લી રહી જાય છે અને મેમરી લીક થાય છે. **Pipeline** એ એક

એવો સેફ મોડ્યુલ ફંક્શન છે જે ડેટા ટ્રાન્સફર પણ કરે છે અને કોઈ પણ એરર આવે તો બધી જ સ્ટ્રીમ્સને ઓટોમેટિકલી સુરક્ષિત રીતે બંધ કરી દે છે.

#### પાઇપલાઇન કોડ:

javascript

```
const { pipeline } = require('stream');
```

```
const fs = require('fs');
```

```
const readStream = fs.createReadStream('source.txt');
```

```
const writeStream = fs.createWriteStream('destination.txt');
```

```
// પાઇપલાઇન દ્વારા ડેટા સેફ ટ્રાન્સફર
```

```
pipeline(readStream, writeStream, (err) => {
```

```
  if (err) {
```

```
    console.error("ટ્રાન્સફર દરમિયાન ભૂલ આવી:", err);
```

```
  } else {
```

```
    console.log("ફાઇલ સફળતાપૂર્વક કોપી થઈ ગઈ!");
```

```
  }
```

```
});
```

Use code with caution.

---

### ૭. Chaining Stream (ચેનિંગ સ્ટ્રીમ)

#### કન્સેપ્ટ:

ચેનિંગ એટલે એક સ્ટ્રીમના આઉટપુટને બીજી સ્ટ્રીમના ઇનપુટ સાથે પાઇપ દ્વારા જોડવું અને આખી એક સાંકળ (Chain) બનાવવી. આનો ઉપયોગ ડેટાને રસ્તામાં જ મોડિફાઇ કે કમ્પ્રેસ (Compress) કરવા માટે થાય છે.

#### ઝીપ ફાઇલ બનાવવાનું ઉદાહરણ:

નીચેનો કોડ એક મોટી ટેક્સ્ટ ફાઇલને સ્ટ્રીમ દ્વારા રીડ કરશે, તેને રસ્તામાં જ **GZIP (Zip)** ફોર્મેટમાં કમ્પ્રેસ કરશે અને ફાઇલ સેવ કરી દેશે:

javascript

```
const fs = require('fs');
```

```
const zlib = require('zlib'); // કમ્પ્રેસ કરવા માટેનું ઇન-બિલ્ટ મોડ્યુલ
```

```
// ૧. રીડ સ્ટ્રીમ બનાવો
const readStream = fs.createReadStream('input.txt');

// ૨. કમ્પ્રેસ કરવાની ટ્રાન્સફોર્મ સ્ટ્રીમ બનાવો
const gzip = zlib.createGzip();

// ૩. રાઇટ સ્ટ્રીમ બનાવો (.gz ફાઇલ માટે)
const writeStream = fs.createWriteStream('input.txt.gz');

// ૪. પેપિંગ (રીડ કરો -> ઝીપ કરો -> રાઇટ કરો)
readStream.pipe(gzip).pipe(writeStream);

console.log("ફાઇલ સફળતાપૂર્વક કમ્પ્રેસ (Zip) થઈ ગઈ છે.");

Use code with caution.
```

---

અહીં **Module 5** ના બધા જ પોઇન્ટ્સ એકદમ ઊંડાણપૂર્વક, આંતરિક લોજિક અને સેટઅપ સાથે પૂરા થાય છે. હવે આપણે આગળ વધીને **Module 6: REST API** માં એકદમ ડીપમાં જઈએ? જો આ મોડ્યુલ (Buffers & Streams) માં કોઈ સવાલ હોય તો પૂછી શકો છો!

ચાલો, હવે આપણે **Module 6: REST API** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ આજના સમયમાં બેકએન્ડ ડેવલપમેન્ટ માટે સૌથી મહત્વનો પાયો છે.

---

## Module 6: REST API (ઊંડાણપૂર્વક અભ્યાસ) [1]

### ૧. RESTful Architecture (RESTful આર્કિટેક્ચર) [1]

#### 📌 વ્યાખ્યા (Definition):

**REST** નું પૂર્ણ નામ **Representational State Transfer** છે. તે બે કમ્પ્યુટર સિસ્ટમ કે એપ્લિકેશન્સ (જેમ કે તમારી React ફ્રન્ટએન્ડ એપ અને Node.js બેકએન્ડ સર્વર) વચ્ચે ઇન્ટરનેટ પર ડેટાની સલામત આપ-લે કરવા માટેના નિયમો અને સિદ્ધાંતોનું એક માળખું (Architecture Style) છે.

#### ⚙️ મુખ્ય સિદ્ધાંતો (Core Principles):

1. **Stateless (સ્ટેટલેસ):** સર્વર ક્યારેય પણ ક્લાયન્ટનું સેશન યાદ રાખતું નથી. ક્લાયન્ટ તરફથી આવતી દરેક રિક્વેસ્ટ એકદમ નવી જ હોય છે, અને તેમાં કામ પૂરું કરવા માટેનો બધો જ ડેટા (જેમ કે ટોકન આઈડી) સાથે જ મોકલવો પડે છે.

2. **Client-Server Separation:** ફ્રન્ટએન્ડ અને બેકએન્ડ બંને એકબીજાથી તદ્દન અલગ હોય છે. તમે બેકએન્ડ બદલો તો ફ્રન્ટએન્ડ પર કોઈ અસર થતી નથી.
3. **Cacheable:** બ્રાઉઝર વારંવાર બદલાતા ન હોય તેવા ડેટાને સેવ કરી શકે છે જેથી સર્વર પર લોડ ન વધે.

---

## ૨. HTTP URI and Methods (HTTP URI અને મેથડ્સ) [1]

REST API માં દરેક વસ્તુને એક સોર્સ (Resource) ગણવામાં આવે છે, અને તેને એક્સેસ કરવા માટે અનોખું વેબ એડ્રેસ હોય છે જેને **URI (Uniform Resource Identifier)** કહેવાય છે.

 **સોર્સ પાથ અને સાચી મેથડ્સનો નકશો:**

**HTTP મેથડ [1] URI પાથ (સોર્સ) [1] સાચો મતલબ (Action)**

|            |                      |                                                            |
|------------|----------------------|------------------------------------------------------------|
| GET [1]    | /api/v1/products     | બધા જ પ્રોડક્ટ્સનું લિસ્ટ મેળવવું (Read) [1]               |
| GET [1]    | /api/v1/products/101 | ફક્ત ૧૦૧ આઈડી વાળા પ્રોડક્ટની ડિટેલ જોવી [1]               |
| POST [1]   | /api/v1/products     | સર્વર પર એક નવો પ્રોડક્ટ ઉમેરવો (Create) [1]               |
| PUT [1]    | /api/v1/products/101 | ૧૦૧ નંબરના પ્રોડક્ટનો આખો ડેટા બદલવો (Update) [1]          |
| DELETE [1] | /api/v1/products/101 | ૧૦૧ નંબરના પ્રોડક્ટને ડેટાબેઝમાંથી કાઢી નાખવો (Delete) [1] |

---

## ૩. RESTful web services (RESTful વેબ સર્વિસીસ) [1]

 **કન્સેપ્ટ:**

જ્યારે આપણે કોઈ એવી વેબ એપ્લિકેશન કે સર્વિસ બનાવીએ જે REST ના તમામ નિયમોનું પાલન કરે છે, ત્યારે તેને **RESTful Web Service** કહેવાય છે. આ સર્વિસ ડેટા ટ્રાન્સફર કરવા માટે મુખ્યત્વે **JSON (JavaScript Object Notation)** ફોર્મેટનો ઉપયોગ કરે છે, જે લાઈટવેઈટ હોય છે અને કોઈપણ પ્રોગ્રામિંગ લેંગ્વેજ (Java, Python, Flutter) તેને સરળતાથી સમજી શકે છે.

---

## ૪. Expose solution as API (સોલ્યુશનને API તરીકે એક્સપોઝ કરવું) [1]

 **કન્સેપ્ટ:**

ધારો કે તમે બેકએન્ડમાં ડેટાબેઝમાંથી ડેટા લાવવાનું એક મોડ્યુલ લોજિક બનાવ્યું છે. હવે આ લોજિકને બહારની દુનિયાના ડેવલપર્સ કે તમારી પોતાની મોબાઇલ ટીમ વાપરી શકે તે માટે રાઉટ્સ લિંક તરીકે કનેક્ટ કરી આપવાની પ્રક્રિયાને **Expose Solution as API** કહેવાય છે.

 **પ્રેક્ટિકલ એક્સપ્રેસ કોડ ઉદાહરણ:**

નીચે આપણે એક લોકલ ડેટા લિસ્ટ બનાવ્યું છે અને તેને કમ્પીટ REST API તરીકે બ્રાઉઝર માટે ખુલ્લું (Expose) કર્યું છે:

javascript

```
const express = require('express');
```

```
const app = express();
```

```
app.use(express.json()); // POST ડેટા વાંચવા માટે જરૂરી મિડલવેર [1]
```

```
// સમી ડેટાબેઝ (લોકલ એરે)
```

```
let users = [
```

```
  { id: 1, name: "રિક્ત" },
```

```
  { id: 2, name: "અજય" }
```

```
];
```

```
// ૧. GET: બધા યુઝર્સ મોકલો [1]
```

```
app.get('/api/v1/users', (req, res) => {
```

```
  res.status(200).json(users);
```

```
});
```

```
// ૨. POST: નવો યુઝર ઉમેરો [1]
```

```
app.post('/api/v1/users', (req, res) => {
```

```
  const newUser = {
```

```
    id: users.length + 1,
```

```
    name: req.body.name
```

```
  };
```

```
  users.push(newUser);
```

```
  res.status(201).json(newUser); // 201 એટલે ડેટા ક્રિએટ થયો
```

```
});
```

```
app.listen(3000, () => console.log("REST API સર્વર ૩૦૦૦ પોર્ટ પર ચાલુ છે."));
```

Use code with caution.

---

**૫. Best practice for REST API solution (શ્રેષ્ઠ પદ્ધતિઓ) [1]**

પ્રોફેશનલ ડેવલપર્સ જ્યારે મોટી કંપનીઓમાં API બનાવે છે, ત્યારે નીચેની ૪ શ્રેષ્ઠ પદ્ધતિઓ (Best Practices) નું ખાસ ધ્યાન રાખે છે:

#### A. API નું વર્ઝન નક્કી કરવું (Versioning)

ભવિષ્યમાં જ્યારે તમારી API માં મોટા ફેરફાર થાય, ત્યારે જૂની એપ્સ બંધ ન થઈ જાય તે માટે URL ની અંદર જ વર્ઝન નંબર લખવો જોઈએ, જેમ કે:

- /api/v1/orders (વર્ઝન ૧)
- /api/v2/orders (વર્ઝન ૨)

#### B. નામો હંમેશા બહુવચનમાં રાખવા (Use Plural Nouns)

URL ની અંદર ક્યારેય ક્રિયાપદ (Verbs) ન લખવા.

- ❌ ખોટી રીત: /api/getNewUser અથવા /api/deleteProduct
- સાચી રીત: GET /api/users અથવા DELETE /api/products/101 [1]

#### C. સાચા HTTP સ્ટેટસ કોડ મોકલવા (HTTP Status Codes)

તમારી API હંમેશા સાચો કોડ રિસ્પોન્સમાં આપતી હોવી જોઈએ:

- **200 OK:** કામ સફળતાપૂર્વક પૂરું થયું.
- **201 Created:** નવો ડેટા સફળતાપૂર્વક બની ગયો.
- **400 Bad Request:** યુઝરે અધૂરો કે ખોટો ડેટા મોકલ્યો છે.
- **401 Unauthorized:** યુઝર લોગઇન નથી અથવા તેની પાસે પરમિશન નથી.
- **404 Not Found:** આ ડેટા સર્વર પર ઉપલબ્ધ નથી.
- **500 Internal Server Error:** તમારા સર્વરના કોડમાં કોઈ ભૂલ કે એરર આવી.

#### D. ફિલ્ટરિંગ અને પેજીનેશન (Filtering & Pagination)

જો ડેટાબેઝમાં ૧૦,૦૦૦ પ્રોડક્ટ્સ હોય, તો એકસાથે બધા જ ડેટા લોડ ન કરવા. ક્લાયન્ટને લિમિટેડ ડેટા આપવા ક્વેરી પેરામીટર વાપરવા, જેમ કે: /api/products?page=1&limit=20

---

અહીં **Module 6** ના તમામ પોઇન્ટ્સ એકદમ વિગતવાર અને પ્રોફેશનલ લોજિક સાથે પૂરા થાય છે.

હવે આપણે આગળ વધીને **Module 7: Callback** માં ઊંડાણપૂર્વક જઈએ? જો આ મોડ્યુલ (REST API) માં કોઈ ડાઉટ હોય તો પૂછી શકો છો!

ચાલો, હવે આપણે **Module 7: Callback** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ જાવાસ્ક્રિપ્ટ અને Node.js ના અસિંક્રોનસ સ્વભાવને સમજવા માટે ખૂબ જ મહત્વનું છે.

---

#### Module 7: Callback (ઊંડાણપૂર્વક અભ્યાસ) [1]

##### ૧. What is Callback (Callback શું છે?) [1]

## ❖ વ્યાખ્યા (Definition):

જાવાસ્ક્રિપ્ટમાં ફંક્શન્સ એ **First-Class Citizens** છે. આનો અર્થ એ થાય કે ફંક્શનને પણ તમે સાદા વેરીએબલની જેમ જ ગમે ત્યાં પાસ કરી શકો છો.

જ્યારે આપણે કોઈ એક ફંક્શનની અંદર આર્ગ્યુમેન્ટ (Parameter) તરીકે બીજા ફંક્શનને પાસ કરીએ, અને પહેલું મુખ્ય ફંક્શન પોતાનું બધું કામ પૂરું કર્યા પછી એ બીજા ફંક્શનને અંદરથી જ ટ્રિગર (Call) કરે, ત્યારે એ બીજા ફંક્શનને **Callback ફંક્શન** કહેવાય છે.

## 📺 એકદમ સાદું અને પ્રેક્ટિકલ ઉદાહરણ:

javascript

// ૧. આ એક સાદું કેલ્ક્યુલેટર ફંક્શન છે, જે ત્રીજા પેરામીટર તરીકે callback લે છે

```
const calculate = (num1, num2, callback) => {  
  const result = num1 + num2;  
  callback(result); // અહીંયા કોલબેક ફંક્શન રન થયું  
};
```

// ૨. આ આપણું અસલી કોલબેક ફંક્શન છે જે જવાબને પ્રિન્ટ કરશે

```
const displayResult = (output) => {  
  console.log("ગણતરીનો જવાબ છે: " + output);  
};
```

// ૩. ફંક્શન કોલ કરતી વખતે displayResult ને આર્ગ્યુમેન્ટ તરીકે મોકલ્યું

```
calculate(10, 20, displayResult);
```

Use code with caution.

---

## ૨. Benefit of Callback (Callback ની ફાયદા) [1]

Node.js માં સિંગલ-થ્રેડેડ મોડેલ હોવાના કારણે કોલબેકના ઘણા મોટા ફાયદાઓ છે:

1. **નોન-બ્લોકિંગ પ્રકૃતિ (Non-blocking nature):** આના લીધે સર્વર કોઈ મોટું કામ (જેમ કે ફાઈલ ડાઉનલોડ કરવી) પૂરું થાય ત્યાં સુધી શાંત બેસી રહેવાના બદલે બીજા કામો ચાલુ રાખી શકે છે.
2. **મેમરી મેનેજમેન્ટ:** જેવું બેકગ્રાઉન્ડ મોટું કામ પૂરું થાય, એટલે કોલબેક ઓટોમેટિકલી જાગી જાય છે અને મેમરી ફ્રી કરીને યુઝરને જવાબ આપી દે છે.

---

## ૩. Asynchronous communications (અસિન્ક્રોનસ કોમ્યુનિકેશન) [1]

## ❖ કન્સેપ્ટ:

જ્યારે Node.js બાહ્ય દુનિયા સાથે વાતચીત કરે છે (જેમ કે કોઈ ત્રીજી પાર્ટીની API માંથી ડેટા લાવવો કે હાર્ડડિસ્ક પર ફાઈલ સેવ કરવી), ત્યારે તે ક્યારેય જવાબની રાહ જોઈને લાઈનમાં ઉભું નથી રહેતું. તે વિનંતી મોકલીને આગળ વધી જાય છે અને વિનંતી સાથે પોતાનું એક **Callback કાર્ડ** જોડી દે છે. જેવો સામેથી ડેટા પાછો આવશે એટલે એ કોલબેક કાર્ડ દ્વારા કોમ્યુનિકેશન પૂરું થાય છે.

## ૪. Block and Non-Blocking (બ્લોકિંગ અને નોન-બ્લોકિંગ) [1]

આ તફાવત જ Node.js ને બીજા બધા બેકએન્ડ ફ્રેમવર્ક કરતા અલગ પાડે છે:

- **Blocking (Synchronous):** જ્યાં સુધી કોડની એક લાઇન પૂરી ન થાય, ત્યાં સુધી નીચેની બીજી લાઇન રન જ નહીં થાય. જો લાઇન-૧ માં મોટી ફાઈલ રીડ થતી હોય, તો આખું સર્વર બાકીના હજારો યુઝર્સ માટે બ્લોક થઈ જશે.
- **Non-Blocking (Asynchronous):** કોડ લાઇન-૧ નું કામ બેકગ્રાઉન્ડના Thread Pool ને સોંપીને તરત જ લાઇન-૨ અને લાઇન-૩ રન કરી દે છે. સર્વર હંમેશા ચાલુ રહે છે અને જ્યારે લાઇન-૧ નું કામ પૂરું થાય ત્યારે કોલબેક તેનો જવાબ સ્ક્રીન પર લાવી દે છે.

## ૫. Standard Callback pattern (સ્ટેન્ડર્ડ કોલબેક પેટર્ન) [1]

Node.js ની બધી જ ઓફિશિયલ ઇન-બિલ્ટ API (જેમ કે fs મોડ્યુલ) એક ચોક્કસ નિયમ પાળે છે, જેને **Error-First Callback Pattern** કહેવામાં આવે છે.

### ⚙ આ પેટર્નના ૨ મુખ્ય નિયમો છે:

1. કોલબેક ફંક્શનનો સૌથી પહેલો પેરામીટર હંમેશા **Error (err)** હોવો જોઈએ. જો પ્રોગ્રામમાં કોઈ ભૂલ આવે તો આમાં ડેટા આવશે.
2. બીજો પેરામીટર હંમેશા અસલી **Result (data)** હોવો જોઈએ. જો કોઈ ભૂલ નહીં હોય તો આમાં સાચો જવાબ આવશે.

### 📄 ઉદાહરણ (Standard Pattern):

javascript

```
const fs = require('fs');
```

```
fs.readFile('student_data.txt', (err, data) => {
```

```
  // નિયમ ૧: સૌથી પહેલા ભૂલ ચેક કરો
```

```
  if (err) {
```

```
    return console.error("ફાઈલ વાંચવામાં ભૂલ આવી છે:", err);
```

```
  }
```

```
// નિયમ ૨: ભૂલ ન હોય તો ડેટા વાપરો  
console.log("ફાઈલનો સાચો ડેટા:\n" + data.toString());  
});
```

Use code with caution.

---

## ૬. Async Flow control Library (અસિંક ફ્લો કંટ્રોલ લાઇબ્રેરી) [1]

### 🔴 Callback Hell (કોલબેક નરક) શું છે?:

જ્યારે આપણે એક પછી એક અસિન્ક્રોનસ કામો કરવાના હોય, ત્યારે કોલબેકની અંદર કોલબેક, અને તેની પણ અંદર ત્રીજો કોલબેક લખવો પડે છે. આનાથી કોડનો આકાર ત્રિકોણ જેવો થઈ જાય છે, જેને ડેવલપર્સ **Callback Hell** અથવા **Pyramid of Doom** કહે છે. આ કોડ વાંચવો અને સમજવો ખૂબ જ અઘરો બની જાય છે.

javascript

```
// Callback Hell નું ઉદાહરણ (વાંચવામાં ખૂબ અઘરું)
```

```
getData(user => {  
  getProfile(user, profile => {  
    getOrders(profile, orders => {  
      getReceipt(orders, receipt => {  
        console.log(receipt);  
      });  
    });  
  });  
});
```

Use code with caution.

### 🔧 ઉકેલ:

જૂના સમયમાં આ ફ્લોને કંટ્રોલ કરવા માટે `async` નામની એક્સટર્નલ લાઇબ્રેરી વાપરવામાં આવતી હતી જે કોડને સીધો રાખવામાં મદદ કરતી હતી. (જોકે અત્યારના આધુનિક Node.js માં આનાથી પણ સરળ ઉપાય તરીકે **Promises** અને **async/await** નો ઉપયોગ થાય છે).

---

## ૭. Executing in parallel (પેરેલલ એક્ઝિક્યુશન) [1]

### 🔴 કન્સેપ્ટ:

ધારો કે આપણે ૩ અલગ-અલગ ડેટાબેઝ ક્વેરી રન કરવી છે અને ત્રણેય ક્વેરી પૂરું થવામાં ૨-૨ સેકન્ડ લે છે.

- જો આપણે એક પછી એક (Series માં) કરીશું, તો કુલ **૬ સેકન્ડ** લાગશે.

- પરંતુ જો આપણે તેને **Parallel (એક જ સમયે એકસાથે)** રન કરી દઈએ, તો ત્રણેય ક્વેરીઓ બેકગ્રાઉન્ડમાં એકસાથે ચાલુ થશે અને માત્ર **૨ સેકન્ડમાં** જ આખો જવાબ આવી જશે. આ પ્રોસેસ કરવા માટે ફ્લો કંટ્રોલ લાઇબ્રેરીના કે પ્રોમિસના Parallel મેકેનિઝમનો ઉપયોગ થાય છે.

---

અહીં **Module 7** ના તમામ પોઇન્ટ્સ એકદમ ડીપમાં અને સાદી ભાષામાં પૂરા થાય છે.

હવે આપણે આગળ વધીને **Module 8: Events** માં આ જ રીતે ઊંડાણપૂર્વક જઈએ? જો આ મોડ્યુલ (Callback) ના કોઈ લોજિકમાં ડાઉટ હોય તો કહી શકો છો!

ચાલો, હવે આપણે **Module 8: Events** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ Node.js ની આર્કિટેક્ચરનો સૌથી પાવરફુલ ભાગ છે, કારણ કે આખી સિસ્ટમ ઇવેન્ટ્સ અને તેના લૂપ પર જ કામ કરે છે.

---

## Module 8: Events (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. What is Events (Events શું છે?)

#### ❖ વ્યાખ્યા (Definition):

Node.js માં **Event** એટલે એપ્લિકેશનની અંદર બનેલી કોઈ ચોક્કસ પ્રવૃત્તિ કે ઘટના.

જ્યારે સર્વર ચાલુ થાય, ત્યારે તે શાંતિથી બેસી રહે છે. જેવી કોઈ ઘટના બને—જેમ કે કોઈ યુઝરે ફાઇલ ઓપન કરી, નેટવર્ક કનેક્શન આવ્યું, અથવા ડેટાબેઝમાંથી ડેટા આવ્યો—એટલે તરત જ સિસ્ટમ જાગી જાય છે અને તે ઘટના સાથે જોડાયેલો કોડ રન કરે છે.

---

### ૨. Events types (ઇવેન્ટ્સના પ્રકાર)

Node.js માં મુખ્ય બે પ્રકારની ઇવેન્ટ્સ હોય છે:

- Core / System Events:** આ એવી ઇવેન્ટ્સ છે જે Node.js ના પોતાના ઇન-બિલ્ટ મોડ્યુલ્સ દ્વારા ઓટોમેટિક જનરેટ થાય છે. જેમ કે જ્યારે ફાઇલ આખી વંચાઈ જાય ત્યારે fs મોડ્યુલ દ્વારા end ઇવેન્ટ ફાયર થાય છે, અથવા નેટવર્ક પર ડેટા આવે ત્યારે data ઇવેન્ટ ટ્રિગર થાય છે.
- Custom Events:** આ એવી ઇવેન્ટ્સ છે જે આપણે પ્રોગ્રામર તરીકે આપણી એપ્લિકેશનની જરૂરિયાત મુજબ આપણી જાતે બનાવીએ છીએ અને ફાયર કરીએ છીએ (દા.ત. યુઝર લોગઇન થાય ત્યારે userLoggedIn ઇવેન્ટ).

---

### ૩. Event Emitter API (ઇવેન્ટ એમિટર API)

#### ❖ કન્સેપ્ટ:

કસ્ટમ ઇવેન્ટ્સ બનાવવા અને તેને હેન્ડલ કરવા માટે Node.js માં એક ઇન-બિલ્ટ મોડ્યુલ આપેલો છે જેને **events** કહેવાય છે. આ મોડ્યુલ આપણને **EventEmitter** નામનો એક પાવરફુલ ક્લાસ આપે છે.

#### 📺 મુખ્ય બે સિન્ટેક્સ:

1. `on(event_name, callback)`: આ ઇવેન્ટ સાંભળવા (Listen કરવા) માટેનું ફંક્શન છે. જ્યારે પણ એ ઘટના બનશે, ત્યારે આની અંદરનું કોલબેક ફંક્શન રન થશે.
2. `emit(event_name, data)`: આ ઇવેન્ટને બળપૂર્વક ટ્રિગર (Fire) કરવા માટેનું ફંક્શન છે. આ લાઇન રન થતાં જ ઇવેન્ટ લિસનર જાગી જશે.

### પ્રેક્ટિકલ ઉદાહરણ:

javascript

```
const EventEmitter = require('events');
```

```
const myEmitter = new EventEmitter();
```

```
// ૧. ઇવેન્ટ લિસનર સેટ કર્યો
```

```
myEmitter.on('paymentSuccess', (amount, user) => {
```

```
  console.log(`ઇવેન્ટ મળી: ${user} એ ₹${amount} નું પેમેન્ટ સફળતાપૂર્વક કર્યું છે.`);
```

```
});
```

```
// ૨. ઇવેન્ટ ફાયર કરી (ધારો કે યુઝરે બટન દબાવ્યું)
```

```
myEmitter.emit('paymentSuccess', 500, 'રહુલ');
```

Use code with caution.

---

## ૪. Multiple event listeners (મલ્ટિપલ ઇવેન્ટ લિસનર્સ)

### કન્સેપ્ટ:

આપણે કોઈ એક જ સિંગલ ઇવેન્ટની પાછળ એક કરતાં વધારે લિસનર્સ (Listeners) ગોઠવી શકીએ છીએ. જ્યારે એ ચોક્કસ ઇવેન્ટ ફાયર થશે, ત્યારે તેની સાથે જોડાયેલા બધા જ ફંક્શન્સ ક્રમબદ્ધ રીતે એક પછી એક ઓટોમેટિક રન થશે.

### ઉદાહરણ:

javascript

```
const EventEmitter = require('events');
```

```
const notification = new EventEmitter();
```

```
// લિસનર ૧: એસએમએસ મોકલો
```

```
notification.on('signup', (email) => console.log(`SMS મોકલ્યો: ${email}`));
```

```
// લિસનર ૨: ઇમેલ મોકલો
```

```
notification.on('signup', (email) => console.log(`Email મોકલ્યો: ${email}`));
```

```
// ઇવેન્ટ ફાયર થતાં જ બંને કામ એકસાથે થશે
```

```
notification.emit('signup', 'user@test.com');
```

Use code with caution.

---

## ૫. Event emitter pattern (ઇવેન્ટ એમિટર પેટર્ન)

### ❖ કન્સેપ્ટ:

મોટા સોફ્ટવેર પ્રોજેક્ટ્સમાં આ એક પ્રખ્યાત ડિઝાઇન પેટર્ન છે. આ પેટર્નનો નિયમ છે કે બે અલગ-અલગ મોટા ક્લાસ કે ફંક્શન્સ ડાયરેક્ટ એકબીજા સાથે કનેક્ટ થવાના બદલે ઇવેન્ટ્સના માધ્યમથી વાતચીત કરે છે. આનાથી કોડ એકદમ લૂઝલી-કપ્લ્ડ (Loosely Coupled) બને છે, એટલે કે જો ભવિષ્યમાં એક મોડ્યુલ બદલાય તો બીજા મોડ્યુલ પર કોઈ ખરાબ અસર પડતી નથી.

---

## ૬. Class methods (ક્લાસ મેથડ્સ)

EventEmitter ક્લાસમાં કામને વધુ સ્માર્ટ બનાવવા માટે કેટલીક ખાસ મેથડ્સ આપેલી છે:

- **once(event\_name, callback):** આ ખૂબ જ મહત્વની મેથડ છે. આનાથી બનેલો લિસનર ઇવેન્ટ બનવા પર ફક્ત એક જ વાર રન થશે. જો એ ઇવેન્ટ બીજી કે ત્રીજી વાર ફાયર થશે, તો આ લિસનર તેને સાંભળશે નહીં (ડીલીટ થઈ જશે).
- **removeListener(event\_name, function\_name):** ચાલુ પ્રોગ્રામમાંથી કોઈ ચોક્કસ લિસનરને હટાવી દેવા માટે આ મેથડ વપરાય છે.

---

## ૭. Event Loop (ઇવેન્ટ લૂપ)

### ❖ સૌથી મુખ્ય કન્સેપ્ટ:

**Event Loop** એ Node.js નો આત્મા છે. તે બેકગ્રાઉન્ડમાં રજ કલાક ચાલતું એક ચક્ર (Infinite Loop) છે.

તેનું મુખ્ય કામ એક જ છે: તે સતત ચેક કરતું રહે છે કે કમ્પ્યુટરના બેકગ્રાઉન્ડમાં (Thread Pool માં) સોંપેલું કોઈ અસિન્ક્રોનસ કામ (જેમ કે ફાઇલ રીડિંગ કે નેટવર્ક રિક્વેસ્ટ) પૂરું થયું છે કે નહીં. જો તે પૂરું થાય, એટલે ઇવેન્ટ લૂપ એ કામના જવાબ અને તેના કોલબેક ફંક્શનને ઉપાડીને મેઇન થ્રેડ (Call Stack) પર રન કરવા માટે મોકલી આપે છે.

---

## ૮. Blocking Event Loop (ઇવેન્ટ લૂપ બ્લોક થવું)

### ❖ જોખમ (The Danger):

Node.js એક જ સિંગલ થ્રેડ પર કામ કરે છે. જો તમે તમારા કોડની અંદર કોઈ ખૂબ જ ભારે કામ મૂકી દો—જેમ કે ૧૦ કરોડ વખત ચાલતું સાદું for લૂપ, મોટું ઇમેજ પ્રોસેસિંગ, કે કોઈ હેવી ફાઇલ કમ્પ્રેસ કરવાનું કામ—તો Node.js નો એ સિંગલ થ્રેડ એ ગણતરીમાં જ અટવાઈ જશે.

આના કારણે બેકએન્ડમાં ચાલતું **Event Loop અટકી જશે (Block થઈ જશે)**. જ્યાં સુધી એ ગણતરી પૂરી નહીં થાય, ત્યાં સુધી ઇવેન્ટ લૂપ ફરી શકશે નહીં અને આખું સર્વર બીજા નવા યુઝર્સ માટે જામ થઈ જશે.

---

### ૯. Escaping Event Loop (ઇવેન્ટ લૂપમાંથી બચવું)

સર્વર ક્યારેય બ્લોક ન થાય અને ઇવેન્ટ લૂપ મુક્ત રીતે ફરતું રહે તે માટે ડેવલપર્સ નીચેની પદ્ધતિઓ વાપરે છે:

1. **setImmediate() અથવા process.nextTick():** જો કોઈ કામ થોડું મોટું હોય, તો તેને તાત્કાલિક રન કરવાના બદલે આ ફંક્શન્સની મદદથી ઇવેન્ટ લૂપના આવતા ચક્ર (Next Tick) પર ધકેલી દેવામાં આવે છે, જેથી કરન્ટ થ્રેડ ફ્રી થઈ જાય.
2. **Child Process કે Worker Threads નો ઉપયોગ:** ખૂબ જ મોટી ગણતરીઓ કે પ્રોસેસિંગ વાળા કામોને મેઇન ઇવેન્ટ લૂપ પર રન કરવાના બદલે **Module 10** માં આપેલા ચાઇલ્ડ પ્રોસેસ મોડેલ પર મોકલી દેવામાં આવે છે, જેથી સર્વર એકદમ સ્મૂથ ચાલતું રહે છે.

---

અહીં **Module 8** ના બધા જ એડવાન્સ પોઇન્ટ્સ એકદમ ઊંડાણપૂર્વક અને સરળ લોજિક સાથે પૂરા થાય છે.

હવે આપણે આગળ વધીને **Module 9: Connecting with Database (MySQL & MongoDB Schema)** માં આ જ રીતે ઊંડાણપૂર્વક જઈએ? જો આ મોડ્યુલ (Events) માં કોઈ પ્રશ્ન હોય તો કહી શકો છો!

Try without personalization

ચાલો, હવે આપણે **Module 9: Connecting with Database**

માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ શીખવાથી તમારો Node.js નો પ્રોડક્ટિવ ડેટાબેઝ સેટઅપ એકદમ પાકો થઈ જશે. આપણે MySQL અને MongoDB બંનેનું આંતરિક લોજિક, સ્કીમા અને કનેક્શન સેટઅપ વિગતવાર સમજીશું.

---

### Module 9: Connecting with Database (ઊંડાણપૂર્વક અભ્યાસ)

#### ૧. Introduction of MySQL (MySQL નો પરિચય)

❖ **વ્યાખ્યા (Definition):**

**MySQL** એ એક ખૂબ જ પ્રખ્યાત, ઓપન-સોર્સ **Relational Database Management System (RDBMS)** છે.

- તે ડેટાને એકદમ ફિક્સ સ્ટ્રક્ચરમાં એટલે કે **ટેબલ્સ (Tables)** ના સ્વરૂપમાં સાચવે છે, જિયાં આડી લાઇનોને રો (Rows) અને ઊભી લાઇનોને કોલમ (Columns) કહેવાય છે.
  - બે ટેબલ વચ્ચે સંબંધો (Relationships - Primary Key અને Foreign Key) નક્કી કરવા માટે અને ડેટાબેઝ સાથે વાતચીત કરવા માટે **SQL (Structured Query Language)** ક્વેરીઝ લખવી પડે છે.
-

## ૨. Connect with MySQL (MySQL સાથે કનેક્ટ કરવું)

Node.js માં પ્રોજેક્શનલ રીતે MySQL વાપરવા માટે **mysql2** પેકેજનો ઉપયોગ થાય છે, જે અસિન્ક્રોનસ Promises ને સપોર્ટ કરે છે.

### ⚙️ કનેક્શન સેટઅપ (Connection Pooling):

મોટા પ્રોજેક્ટ્સમાં સાદું કનેક્શન બનાવવાના બદલે **Pool** બનાવવામાં આવે છે. Pool નો ફાયદો એ છે કે તે એકસાથે ઘણા બધા કનેક્શન્સ ઓપન રાખે છે, જેથી દરેક નવી રિક્વેસ્ટ વખતે નવું કનેક્શન બનાવવાનો સમય બચે છે.

ચાલો, હવે આપણે **Module 9: Connecting with Database (MySQL & MongoDB)** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ. આ મોડ્યુલ બેકએન્ડ ડેવલપમેન્ટમાં ડેટાને કાયમી (Persist) કરવા માટે સૌથી મહત્વનો ભાગ છે.

---

## Module 9: Connecting with Database (ઊંડાણપૂર્વક અભ્યાસ)

### ૧. Introduction of MySQL (MySQL નો પરિચય)

#### 📌 વ્યાખ્યા (Definition):

**MySQL** એ એક ખૂબ જ પ્રખ્યાત ઓપન-સોર્સ **Relational Database Management System (RDBMS)** છે.

- તે ડેટાને ટેબલ્સ (Tables) ના સ્વરૂપમાં એટલે કે આડી લાઇન (Rows/Records) અને ઊભી લાઇન (Columns/Fields) માં સ્ટોર કરે છે.
- બે ટેબલ્સ વચ્ચે સંબંધ નક્કી કરવા માટે તેમાં પ્રાઇમરી કી (Primary Key) અને ફોરેન કી (Foreign Key) નો ઉપયોગ થાય છે. ડેટા મેળવવા કે લખવા માટે આપણે **SQL (Structured Query Language)** નો ઉપયોગ કરવો પડે છે.

---

## ૨. Connect with MySQL (MySQL સાથે કનેક્ટ કરવું)

Node.js માં MySQL ડેટાબેઝ સાથે વાતચીત કરવા માટે **mysql2** નામનું પેકેજ સૌથી બેસ્ટ અને ફાસ્ટ ગણાય છે.

bash

# ટર્મિનલમાં મોડ્યુલ ઇન્સ્ટોલ કરો

npm install mysql2

Use code with caution.

### 📄 કનેક્શન અને ક્વેરી રન કરવાનો કોડ:

javascript

```
const mysql = require('mysql2');
```

```
// ૧. ડેટાબેઝ સાથે કનેક્શનનું માળખું સેટ કરો
const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',    // તમારું MySQL યુઝરનેમ
  password: 'password', // તમારો પાસવર્ડ
  database: 'my_company' // ડેટાબેઝનું નામ
});

// ૨. કનેક્શન ચાલુ કરીને SQL ક્વેરી રન કરો
connection.query(
  'SELECT * FROM users WHERE age > ?', [18], // SQL ક્વેરી
  (err, results, fields) => {
    if (err) return console.error("MySQL એરર:", err);

    console.log("યુઝર્સનો ડેટા આવી ગયો:");
    console.log(results); // આઉટપુટ એરે ઓફ ઓબ્જેક્ટ્સમાં મળશે
  }
);

Use code with caution.
```

---

### ૩. Introduction of MongoDB (MongoDB નો પરિચય)

#### ❖ વ્યાખ્યા (Definition):

MongoDB એ એક આધુનિક, ઓપન-સોર્સ NoSQL (Not Only SQL) Document-Oriented Database છે.

- તે MySQL ની જેમ ટેબલ અને રો (Row) માં ડેટા સેવ નથી કરતું.
- તે ડેટાને JSON જેવા ફ્લેક્સિબલ ડોક્યુમેન્ટ્સ (જેને ટેકનિકલી BSON કહેવાય છે) ના સ્વરૂપમાં સ્ટોર કરે છે. આના લીધે ડેટાબેઝ ખૂબ જ ફાસ્ટ બને છે અને સ્કેલ (Scale) કરવો સહેલો પડે છે.

---

### ૪. Features of MongoDB (MongoDB ની ખાસિયતો)

1. **Schema-less (ફ્લેક્સિબલ માળખું):** એક જ ક્વેક્શનની અંદર રહેલા બે ડોક્યુમેન્ટ્સ તદ્દન અલગ ડેટા અને અલગ ફિલ્ડ્સ સાચવી શકે છે. અગાઉથી કોઈ ફિક્સ કોલમ નક્કી કરવાની જરૂર નથી.

2. **High Performance:** તે ઇન-મેમરી ઇન્ડેક્સિંગ સપોર્ટ કરે છે, જેથી લાખો રેકૉર્ડ્સમાંથી ડેટા શોધવાનું કામ મિલીસેકન્ડ્સમાં થાય છે.
3. **Easy Scaling (Horizontal Scaling):** શાર્ડિંગ (Sharding) ટેકનોલોજીની મદદથી ડેટાને એક કરતાં વધુ સર્વર્સ પર વહેંચવો ખૂબ જ આસાન છે.

---

#### ૫. Connect with MongoDB (MongoDB સાથે કનેક્ટ કરવું)

Node.js માં મોંગોડીબી સાથે એકદમ ઓર્ગનાઇઝ્ડ અને પ્રોફેશનલ રીતે કામ કરવા માટે **mongoose** નામના ODM (Object Data Modeling) પેકેજનો ઉપયોગ થાય છે.

bash

```
# ટર્મિનલમાં મોંગૂસ ઇન્સ્ટોલ કરો
```

```
npm install mongoose
```

Use code with caution.

##### કનેક્શન કોડ:

javascript

```
const mongoose = require('mongoose');
```

```
// લોકલ મોંગોડીબી સર્વર સાથે કનેક્ટ કરવું
```

```
mongoose.connect('mongodb://localhost:27017/ecom_db')
```

```
.then(() => console.log("MongoDB સાથે કનેક્શન સફળ થયું!"))
```

```
.catch((err) => console.error("MongoDB કનેક્શન નિષ્ફળ:", err));
```

Use code with caution.

---

#### ૬. Defining a schema (સ્કીમા વ્યાખ્યાયિત કરવી)

##### કન્સેપ્ટ:

મોંગોડીબી પોતે સ્કીમા-લેસ (નિયમ વગરનું) છે, પણ જો યુઝર ગમે તેવો ડેટા ડેટાબેઝમાં નાખી દે તો એપ્લિકેશન તૂટી શકે છે. આથી, આપણે આપણા Node.js કોડની અંદર Mongoose.Schema નો ઉપયોગ કરીને ડેટાનું એક ફિક્સ ફોર્મેટ અને વેલિડેશન નિયમો નક્કી કરીએ છીએ, તેને **Schema** કહેવાય છે.

##### પ્રોફેશનલ સ્કીમા ઉદાહરણ:

javascript

```
const mongoose = require('mongoose');
```

```
const productSchema = new mongoose.Schema({
```

```
title: {
  type: String,
  required: [true, "પ્રોડક્ટનું નામ આપવું ફરજિયાત છે"], // Validation Rule
  trim: true
},
price: {
  type: Number,
  required: true,
  min: [0, "કિંમત ૦ થી ઓછી ન હોઈ શકે"]
},
category: {
  type: String,
  default: "General"
},
createdAt: {
  type: Date,
  default: Date.now
}
});
```

Use code with caution.

---

## ૭. Defining a Model (મોડેલ વ્યાખ્યાયિત કરવું)

### ◆ કન્સેપ્ટ:

સ્કીમા એ ફક્ત એક નકશો (Design) છે. એ નકશાના આધારે ડેટાબેઝના કલેક્શન (Collection) સાથે અસલી ઓપરેશન્સ કરવા માટે—જેમ કે નવો ડેટા સેવ કરવો, શોધવો કે ડીલીટ કરવો—આપણે જે સ્પેશિયલ જાવાસ્ક્રિપ્ટ ક્લાસ ઓબ્જેક્ટ બનાવીએ છીએ, તેને **Model** કહેવાય છે.

### 🖥 મોડેલ બનાવવું અને ડેટા સેવ કરવાનું ઉદાહરણ:

javascript

```
// ૧. સ્કીમા ઉપરથી મોડેલ બનાવો (ડેટાબેઝમાં ઓટોમેટિક 'products' નામનું કલેક્શન બનશે)
const Product = mongoose.model('Product', productSchema);
```

```
// ૨. મોડેલનો ઉપયોગ કરીને નવો ડેટા ક્રિએટ કરવાનો ફંક્શન
const createNewProduct = async () => {
  try {
    const item = new Product({
      title: "વાયરલેસ માઉસ",
      price: 499,
      category: "Electronics"
    });

    // ડેટાબેઝમાં સેવ કરો
    const savedItem = await item.save();
    console.log("પ્રોડક્ટ ડેટાબેઝમાં સેવ થઈ ગયો છે:", savedItem);
  } catch (err) {
    console.error("ડેટા સેવ ન થયો, વેલિડેશન એરર:", err.message);
  }
};
```

createNewProduct();

Use code with caution.

---

અહીં **Module 9** ના બધા જ ડેટાબેઝ કન્સેપ્ટ્સ એકદમ ઊંડાણપૂર્વક અને લાઈવ વેલિડેશન લોજિક સાથે પૂરા થાય છે.

હવે આપણે સિલેબસના છેલ્લા અને સૌથી મહત્વના **Module 10: External Processes and Services (Child Processes)** માં આ જ રીતે ઊંડાણપૂર્વક આગળ વધીએ? જો આ મોડ્યુલ (Database) માં કોઈ ડાઉન હોય તો કહી શકો છો!

ચાલો, હવે આપણે તમારા સિલેબસના છેલ્લા અને સૌથી એડવાન્સ મોડ્યુલ

**Module 10: External Processes and Services** માં એકદમ ઊંડાણપૂર્વક (Deep Detail) જઈએ.

આ મોડ્યુલ Node.js ના સિંગલ-થ્રેડેડ મોડેલની મર્યાદાઓને તોડીને કમ્પ્યુટરના મલ્ટી-કોર પ્રોસેસર (Multi-core Processor) નો પૂરેપૂરો ઉપયોગ કેવી રીતે કરવો તે સમજાવે છે.

---

**Module 10: External Processes and Services (ઊંડાણપૂર્વક અભ્યાસ)**

## ૧. What is processes (પ્રોસેસ એટલે શું?)

### ❖ વ્યાખ્યા (Definition):

જ્યારે કમ્પ્યુટરમાં કોઈ પણ સોફ્ટવેર કે પ્રોગ્રામ રન (Execute) થઈ રહ્યો હોય, ત્યારે ઓપરેટિંગ સિસ્ટમ (Windows/Mac/Linux) તેને ચાલવા માટે જે ખાસ મેમરી (RAM), પ્રોસેસરનો સમય અને એક અનોખો નંબર આપે છે, તેને **Process (પ્રક્રિયા)** કહેવાય છે.

- **PID (Process ID):** દરેક પ્રોસેસને પોતાનો એક આગવો નંબર મળે છે જેને PID કહેવાય છે.
- **Node.js Process:** જ્યારે તમે ટર્મિનલમાં `node app.js` લખો છો, ત્યારે ઓપરેટિંગ સિસ્ટમ બેકગ્રાઉન્ડમાં એક મુખ્ય પ્રોસેસ શરૂ કરે છે. Node.js માં આ પ્રોસેસની બધી જ વિગતો મેળવવા માટે `child_process` મોડ્યુલ **process** ઓબ્જેક્ટ આપેલો છે.
- **ઉદાહરણ:** `console.log(process.pid);` લખવાથી તમારા ચાલુ પ્રોગ્રામનો PID જાણી શકાય છે.

---

## ૨. Spawning Child process (ચાઇલ્ડ પ્રોસેસ સ્પોન કરવી)

### ❖ ઊંડાણપૂર્વક કન્સેપ્ટ (Why do we need Child Process?):

અગાઉ **Module 8** માં આપણે જોયું કે Node.js **Single-Threaded** છે. જો આપણે કોઈ ખૂબ જ ભારે ગણતરી (દા.ત. વીડિયો કન્વર્ટ કરવો કે લાખો લાઈનનો ડેટા સોર્ટ કરવો) મેઇન થ્રેડ પર કરીએ, તો આપણું આખું સર્વર બ્લોક થઈ જાય છે.

આ સમસ્યાના ઉકેલ માટે Node.js આપણને ઓપરેટિંગ સિસ્ટમમાં પોતાના જેવી જ અથવા કમ્પ્યુટરના અન્ય કમાન્ડ્સ રન કરવા માટે નવી **Child Process (બાળ પ્રક્રિયા)** બનાવવાની મંજૂરી આપે છે. આના માટે Node.js માં `child_process` મોડ્યુલ આપેલો છે.

### ⚙️ `spawn()` વિરુદ્ધ `exec()` (The Core Distinction):

ચાઇલ્ડ પ્રોસેસ બનાવવા માટે મુખ્ય બે રીતો છે:

1. **`exec()`:** આ મેથડ આખા આઉટપુટને પૂરું થવાની રાહ જોવે છે અને છેલ્લે બધો જ ડેટા એકસાથે બફર (Buffer) તરીકે પાછો આપે છે. આ નાના કમાન્ડ્સ માટે યોગ્ય છે (લિમિટ: 200KB).
2. **`spawn()`:** આ મેથડ ડેટાને સ્ટ્રીમ્સ (Streams) ના સ્વરૂપમાં ટુકડે-ટુકડે (Chunks માં) લાઈવ પાછો આપે છે. ખૂબ જ મોટા ડેટા કે લાંબા સમય સુધી ચાલતા કમાન્ડ્સ માટે આ મેથડ બેસ્ટ ગણાય છે.

### 📄 `spawn()` નો પ્રેક્ટિકલ કોડ ઉદાહરણ:

નીચેનો કોડ તમારા કમ્પ્યુટરની સિસ્ટમમાં લિનક્સ/મેક માટે `ls -lh` અથવા વિન્ડોઝ માટે `dir` નો કમાન્ડ બેકગ્રાઉન્ડમાં ચાઇલ્ડ પ્રોસેસ તરીકે રન કરશે:

```
javascript
```

```
const { spawn } = require('child_process');
```

```
// વિન્ડોઝ માટે 'dir' અથવા Linux/Mac માટે 'ls' કમાન્ડ સ્પોન (Spawn) કરો
```

```
// અહીં આપણે લિનક્સ/મેક ના ઉદાહરણ તરીકે 'ls' અને તેનો આર્ગ્યુમેન્ટ '-lh' લીધો છે
```

```
const child = spawn('ls', ['-lh']);
```

```
// ૧. યાઇલ્ડ પ્રોસેસમાંથી જેવો લાઈવ ડેટા આવવાનો શરૂ થાય (Readable Stream)
```

```
child.stdout.on('data', (data) => {  
  console.log(`યાઇલ્ડ પ્રોસેસમાંથી લાઈવ ડેટા મળ્યો: \n${data}`);  
});
```

```
// ૨. જો યાઇલ્ડ પ્રોસેસ રન થતી વખતે કોઈ ભૂલ કે એરર આવે
```

```
child.stderr.on('data', (data) => {  
  console.error(`યાઇલ્ડ પ્રોસેસમાં ભૂલ આવી: ${data}`);  
});
```

```
// ૩. જ્યારે યાઇલ્ડ પ્રોસેસ પોતાનું કામ પૂરું કરીને બંધ થાય
```

```
child.on('close', (code) => {  
  console.log(`યાઇલ્ડ પ્રોસેસ સફળતાપૂર્વક બંધ થઈ. એક્ઝિટ કોડ: ${code}`);  
});
```

Use code with caution.

---

### ૩. Create and kill processes (પ્રોસેસ બનાવવી અને બંધ કરવી)

મોટા અને પ્રોફેશનલ બેકએન્ડ આર્કિટેક્ચરમાં, આપણે યાઇલ્ડ પ્રોસેસ માત્ર બનાવતા જ નથી, પણ તેને કંટ્રોલ પણ કરવી પડે છે. ક્યારેક એવું બને કે કોઈ બેકએન્ડ પ્રક્રિયા (જેમ કે કોઈ ફાઇલ ડાઉનલોડિંગ) અટવાઈ ગઈ હોય અથવા અનંત લૂપ (Infinite Loop) માં જતી રહી હોય, ત્યારે તેને બંધ કરવી ફરજિયાત બને છે.

#### પ્રોસેસ કિલ (kill) કરવાનું ઉદાહરણ:

Node.js માં આપણે યાઇલ્ડ પ્રોસેસના ઓબ્જેક્ટ પર `.kill()` મેથડ ચલાવીને તેને ઓપરેટિંગ સિસ્ટમ લેવલ પરથી ફોર્સફુલી બંધ કરી શકીએ છીએ.

javascript

```
const { spawn } = require('child_process');
```

```
// એક લાંબો સમય ચાલતી પ્રોસેસ બનાવો (દા.ત. ૧૦ કલાક સુધી ચાલતો કોઈ કમાન્ડ)
```

```
const longProcess = spawn('ping', ['google.com']);
```

```

console.log(`નવી ચાઇલ્ડ પ્રોસેસ શરૂ થઈ. PID: ${longProcess.pid}`);

// ધારો કે ૫ સેકન્ડ પછી આપણે આ પ્રોસેસને બંધ કરી દેવી છે
setTimeout(() => {
  console.log("પ્રોસેસ ઘણો સમય લઈ રહી છે, તેને બંધ (Kill) કરીએ છીએ...");

  // `kill()` મેથડ ઓપરેટિંગ સિસ્ટમને SIGTERM સિગ્નલ મોકલીને પ્રોસેસ સાફ કરી દેશે
  longProcess.kill();

}, 5000);

// પ્રોસેસ બંધ થઈ કે નહીં તેનું વેરિફિકેશન
longProcess.on('close', (code) => {
  console.log(`પ્રોસેસને મારી નાખવામાં આવી છે. સ્ટેટસ કોડ: ${code}`);
});

Use code with caution.

```

### આ મોડ્યુલના મુખ્ય ફાયદા:

1. **No Core Blockage:** મેઇન વેબ સર્વર ક્યારેય હેંગ થતું નથી, ભલે પાછળથી ગમે તેટલું ભારે પ્રોસેસિંગ ચાલતું હોય.
2. **Multi-tasking:** તમે Node.js માં બેઠા-બેઠા જ તમારા કમ્પ્યુટરના અન્ય સોફ્ટવેર્સ (જેમ કે Python સ્ક્રિપ્ટ્સ અથવા C++ પ્રોગ્રામ્સ) ને કમાન્ડ આપીને રન કરાવી શકો છો અને તેનો જવાબ મેળવી શકો છો.

---

અહીં મોકલેલા સિલેબસનો છેલ્લો અને **૧૦મો મોડ્યુલ પણ એકદમ ઊંડાણપૂર્વક, આંતરિક લોજિક અને કમ્પ્લીટ પ્રેક્ટિકલ સિન્ટેક્સ સાથે પૂરો થાય છે [1].**

તમે શરૂઆતથી લઈને અંત સુધીના તમામ મોડ્યુલ અને તેના પેટા-ટોપિક્સ ખૂબ જ સરસ રીતે અને ઝીણવટપૂર્વક શીખી લીધા છે!

હવે તમારો **Node.js નો આખો સિલેબસ થિયરી અને પ્રેક્ટિકલ લોજિક સાથે કમ્પ્લીટ** થઈ ગયો છે. આગળ વધવા માટે તમે શું કરવા માંગો છો:

1. આ આખા સિલેબસ પરથી તમારો ટેસ્ટ લેવા માટે એક **ઇનલાઇન ક્વિઝ (Inline Quiz)** સેટ કરું?
2. **Express અને MongoDB** નો ઉપયોગ કરીને એક મોટો ફૂલ-સ્ટેક **લાઇવ પ્રોજેક્ટ** કોડિંગ સાથે શરૂ કરીએ?

આપણે અત્યાર સુધી શીખેલા તમામ મોડ્યુલ્સ (Express Framework, Middleware, REST API, Callbacks, Events, અને Database Mongoose Schema) નો ઉપયોગ કરીને એક

**રિયલ-વર્લ્ડ બેકએન્ડ પ્રોજેક્ટ** બનાવીશું.

આપણે એક **E-Commerce Product Management API** (પ્રોડક્ટ મેનેજમેન્ટ સિસ્ટમ) બનાવીશું. આ પ્રોજેક્ટમાં ડેટાબેઝ તરીકે **MongoDB** અને સર્વર માટે **Express** નો ઉપયોગ કરીશું.

પ્રોજેક્ટના આયોજન માટે તમારા ફોલ્ડરમાં નીચે મુજબની ફાઇલો કમબદ્ધ રીતે સેટ કરો:

---

## ૧. package.json (પ્રોજેક્ટ કન્ફિગરેશન)

તમારા ખાલી ફોલ્ડરમાં ટર્મિનલ ઓપન કરી `npm init -y` કરો, ત્યારબાદ `npm i express mongoose` ઇન્સ્ટોલ કરીને આ ફાઇલ બનાવો:

```
json
{
  "name": "ecommerce-api",
  "version": "1.0.0",
  "description": "E-Commerce backend product API solution",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.19.2",
    "mongoose": "^8.3.1"
  }
}
```

Use code with caution.

---

## ૨. db.js (ડેટાબેઝ કનેક્શન અને ઇવેન્ટ હેન્ડલિંગ)

**Module 8 (Events)** અને **Module 9 (Database)** નો ઉપયોગ કરીને ડેટાબેઝ કનેક્શન સેટ કરીએ:

```
javascript
const mongoose = require('mongoose');
```

```
const connectDB = async () => {
  try {
    // MongoDB સાથે લોકલ કનેક્શન (Module 9)
    const conn = await mongoose.connect('mongodb://localhost:27017/ecommerce_db');
    console.log('MongoDB કનેક્ટ થઈ ગયું: ${conn.connection.host}');
  } catch (error) {
    console.error('ડેટાબેઝ એરર: ${error.message}');
    process.exit(1); // જો કનેક્શન ફેલ થાય તો પ્રોસેસ બંધ કરો (Module 10)
  }
};

module.exports = connectDB;

Use code with caution.
```

---

### 3. productModel.js (Mongoose સ્કીમા અને મોડેલ)

**Module 9** મુજબ ડેટા વેલિડેશન સાથે પ્રોડક્ટનો નકશો (Schema) તૈયાર કરીએ:

javascript

```
const mongoose = require('mongoose');

// ૧. સ્કીમા વ્યાખ્યાયિત કરવી (Defining a Schema)
const productSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, "પ્રોડક્ટનું નામ આપવું ફરજિયાત છે"],
    trim: true
  },
  price: {
    type: Number,
    required: [true, "કિંમત આપવી ફરજિયાત છે"],
    min: [0, "કિંમત ૦ થી ઓછી ન હોઈ શકે"]
  }
});
```

```
    },  
    category: {  
      type: String,  
      required: true  
    },  
    inStock: {  
      type: Boolean,  
      default: true  
    }  
  }, { timestamps: true });
```

// ૨. મોડેલ વ્યાખ્યાયિત કરવું (Defining a Model)

```
const Product = mongoose.model('Product', productSchema);
```

```
module.exports = Product;
```

Use code with caution.

---

#### ૪. server.js (મુખ્ય સર્વર, મિડલવેર અને REST API રાઉટ્સ)

**Module 4 (Express)** અને **Module 6 (REST API)** નો ઉપયોગ કરીને તમામ CRUD ઓપરેશન્સ આ ફાઈલમાં ભેગા કરીએ:

javascript

```
const express = require('express');  
const connectDB = require('./db');  
const Product = require('./productModel');
```

```
const app = express();
```

```
const PORT = 3000;
```

```
// ડેટાબેઝ કનેક્ટ કરો
```

```
connectDB();
```

```
// Route Middleware (Module 4) - JSON ડેટા રીડ કરવા માટે
```

```
app.use(express.json());
```

```
// લોગ રાખવા માટેનું ઝવોબલ મિડલવેર (Route Middleware)
```

```
app.use((req, res, next) => {
```

```
  console.log(`[LOG] ${new Date().toISOString()} - ${req.method} ${req.url}`);
```

```
  next();
```

```
});
```

```
// --- REST API ROUTES (Module 6) ---
```

```
// ૧. GET: બધા જ પ્રોડક્ટ્સનું લિસ્ટ મેળવવા (Read)
```

```
app.get('/api/v1/products', async (req, res) => {
```

```
  try {
```

```
    const products = await Product.find();
```

```
    res.status(200).json(products);
```

```
  } catch (err) {
```

```
    res.status(500).json({ error: err.message });
```

```
  }
```

```
});
```

```
// ૨. POST: નવો પ્રોડક્ટ ઉમેરવા (Create)
```

```
app.post('/api/v1/products', async (req, res) => {
```

```
  try {
```

```
    const newProduct = new Product(req.body);
```

```
    const savedProduct = await newProduct.save();
```

```
    res.status(201).json(savedProduct); // 201 Created
```

```
  } catch (err) {
```

```
    res.status(400).json({ error: err.message }); // 400 Bad Request
```

```
  }
```

```
});
```

// ૩. PUT: પોસ્ટ પ્રોડક્ટ અપડેટ કરવા (Update)

```
app.put('/api/v1/products/:id', async (req, res) => {  
  try {  
    const updatedProduct = await Product.findByIdAndUpdate(  
      req.params.id,  
      req.body,  
      { new: true, runValidators: true }  
    );  
    if (!updatedProduct) return res.status(404).json({ message: "પ્રોડક્ટ ન મળ્યો" });  
    res.status(200).json(updatedProduct);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
});
```

// ૪. DELETE: પ્રોડક્ટ કાઢી નાખવા (Delete)

```
app.delete('/api/v1/products/:id', async (req, res) => {  
  try {  
    const deletedProduct = await Product.findByIdAndDelete(req.params.id);  
    if (!deletedProduct) return res.status(404).json({ message: "પ્રોડક્ટ ઓવરેડી ડીલીટ થઈ ગયો છે" });  
    res.status(200).json({ message: "પ્રોડક્ટ સફળતાપૂર્વક ડીલીટ થયો" });  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
});
```

// સર્વર ચાલુ કરો

```
app.listen(PORT, () => {  
  console.log(`બેકએન્ડ સર્વર http://localhost:${PORT} પર વાઈવ છે.`);  
});
```

});

Use code with caution.

### 🚀 પ્રોજેક્ટ રન કરવાની રીત:

1. તમારા કમ્પ્યુટરમાં **MongoDB Compass** અથવા સર્વર બેકએન્ડમાં ચાલુ હોવું જોઈએ.
2. ટર્મિનલમાં લખો: `npm start`
3. તમે **Postman** અથવા બ્રાઉઝરની મદદથી `http://localhost:3000/api/v1/products` લિંક પર જઈને ડેટા ક્વિએટ કે રીડ કરીને ચેક કરી શકો છો.

આ આપણો એક કમ્પ્લીટ પ્રોડક્ટ મેનેજમેન્ટ બેકએન્ડ પ્રોજેક્ટ તૈયાર થઈ ગયો છે! જો તમને આ કોડ સમજવામાં કે રન કરવામાં કોઈ તકલીફ હોય તો જણાવો, અથવા જો પ્રોજેક્ટ રેડી હોય તો આપણે આખા સિલેબસનો **આખરી પ્રેક્ટિસ ટેસ્ટ (Quiz)** શરૂ કરીએ?

તમે મોકલેલા સિલેબસમાં Node.js, Express અને MongoDB નો પાયો (Fundamentals) ખૂબ જ સરસ રીતે કવર થઈ ગયો છે. પરંતુ, રિયલ-વર્લ્ડમાં જ્યારે તમે કોઈ પ્રોફેશનલ કે મોટો લાઈવ પ્રોજેક્ટ બનાવો, ત્યારે આ સિલેબસ સિવાય પણ કેટલાક

**ખૂબ જ મહત્વના એડવાન્સ ટોપિક્સ** શીખવા જરૂરી છે.

આ સિવાય કયા-કયા ટોપિક્સ ઘટે છે તેનું આખું લિસ્ટ નીચે મુજબ છે:

### ૧. Node.js માં ઘટતા એડવાન્સ ટોપિક્સ (Node.js Missing Concepts)

- **Worker Threads:** ચાઇલ્ડ પ્રોસેસ (Module 10) ની જેમ જ, સિંગલ થ્રેડની અંદર જ મલ્ટી-થ્રેડિંગ જેવું કામ કરવા અને ભારે CPU ગણતરીઓ મેઇન ઇવેન્ટ લૂપ બ્લોક કર્યા વગર કરવા માટે આ ખૂબ જરૂરી છે.
- **Crypto મોડ્યુલ:** યુઝરના પાસવર્ડને ડાયરેક્ટ ડેટાબેઝમાં સેવ કરવાના બદલે તેને હેશેડ (Hashed / Encrypt) કરવા માટે Node.js નું ઇન-બિલ્ટ સિક્યોરિટી મોડ્યુલ.
- **Clusters:** તમારા કમ્પ્યુટરના બધા જ CPU Cores નો ઉપયોગ કરીને સર્વરની ક્ષમતા અને સ્પીડ વધારવા માટે.

### ૨. Express.js માં ઘટતા એડવાન્સ ટોપિક્સ (Express.js Missing Concepts)

- **Authentication & Authorization (JWT):** યુઝર સાઇન-ઇન (Login) થાય ત્યારે તેના સેશનને સુરક્ષિત રાખવા માટે **JSON Web Token (JWT)** નો ઉપયોગ કરવો.
- **Validation (Joi / Express-Validator):** API માં ખોટો ડેટા ન આવે તે માટે રિક્વેસ્ટના બોડી ડેટા (req.body) ને ચેક કરવા માટેનું પ્રોફેશનલ મોડ્યુલ.
- **Error Handling Middleware:** આખી એપ્લિકેશનમાં ગમે ત્યાં એરર આવે તો આખું સર્વર કેશ થવાના બદલે તેને એક જ જગ્યાએ સેન્ટ્રલાઇઝ્ડ હેન્ડલ કરવાની સિસ્ટમ.
- **CORS (Cross-Origin Resource Sharing):** તમારા ફ્રન્ટએન્ડ (જેમ કે React કે HTML પેજ) માંથી બેકએન્ડ API એક્સેસ કરતી વખતે આવતી સિક્યોરિટી એરરને કોર્ડિંગ દ્વારા કેવી રીતે મેનેજ કરવી.

### ૩. MongoDB માં ઘટતા એડવાન્સ ટોપિક્સ (MongoDB Missing Concepts)

- **Aggregation Framework:** ડેટાબેઝમાંથી એડવાન્સ ફિલ્ટરિંગ, ગ્રુપિંગ (Group By), અને ગણતરીઓ (જેમ કે ટોટલ સેલ્સ કાઢવું) કરવા માટેની ખૂબ જ પાવરફુલ સિસ્ટમ.
- **Indexing:** લાખો ડેટામાંથી સેકન્ડના હજારમા ભાગમાં ડેટા શોધવા માટે પર્ફોર્મન્સ બુસ્ટ કરવાની રીત.
- **Mongoose Middleware (Hooks):** ડેટાબેઝમાં ડેટા સેવ થાય તેની બરાબર પહેલાં કે પછી કોઈ કસ્ટમ ફંક્શન રન કરવું (જેમ કે pre('save') માં પાસવર્ડ ઇન્ક્રિપ્ટ કરવો).
- **Population (Relationships):** બે અલગ-અલગ કલેક્શન્સ (જેમ કે Users અને Orders) ને એકબીજા સાથે જોડીને જોઈન્ટ ડેટા લાવવો.

### ૪. રીયલ-વર્લ્ડ પ્રોજેક્ટ આર્કિટેક્ચર (Professional Practices)

- **MVC Pattern (Model-View-Controller):** મોટો પ્રોજેક્ટ બનાવતી વખતે બધો કોડ એક જ ફાઈલમાં લખવાના બદલે રાઉટ્સ, કંટ્રોલર્સ અને મોડેલ્સને અલગ-અલગ ફોલ્ડરમાં ગોઠવવાની પ્રોફેશનલ રીત.
- **Environment Variables (.env):** ડેટાબેઝના પાસવર્ડ કે સિક્રેટ કી જેવા સેન્સિટિવ ડેટાને કોડમાં હાર્ડકોડ કરવાના બદલે છુપાવીને રાખવા માટે dotenv પેકેજનો ઉપયોગ.

મેં આ લિસ્ટમાં લાઈવ માર્કેટમાં વપરાતા એડવાન્સ ટોપિક્સ સમજાવ્યા છે. શું તમારે આમાંથી કોઈ ચોક્કસ ટોપિક (જેમ કે **JWT લોગિન સિસ્ટમ** કે **MVC ફોલ્ડર સ્ટ્રક્ચર**) ને આપણા પ્રોજેક્ટમાં ઉમેરીને તેને વધુ પ્રોફેશનલ બનાવવો છે?

ચાલો, આપણે આપણા ગયા પ્રોજેક્ટને એકદમ પ્રોફેશનલ લેવલ પર લઈ જઈએ. આ માટે આપણે પ્રોજેક્ટમાં બે સૌથી મહત્વની વસ્તુઓ ઉમેરીશું:

1. **MVC Pattern (Model-View-Controller):** બધો કોડ એક જ ફાઈલમાં લખવાના બદલે રાઉટ્સ, કંટ્રોલર્સ અને મોડેલ્સના અલગ-અલગ ફોલ્ડર બનાવીશું.
2. **Environment Variables (.env):** ડેટાબેઝના પાસવર્ડ અને પોર્ટ નંબરને કોડમાં હાર્ડકોડ કરવાના બદલે સુરક્ષિત ફાઈલમાં છુપાવીને રાખીશું.

સૌથી પહેલા તમારા કમ્પ્યુટરમાં પ્રોજેક્ટનું નવું ફોલ્ડર સ્ટ્રક્ચર આ રીતે ગોઠવો:

#### 📁 પ્રોજેક્ટ ફોલ્ડરનું માળખું (MVC Structure)

text

ecommerce-mvc/

├── .env

├── db.js

├── server.js

├── models/

| └── productModel.js

└── controllers/

```
| └─ productController.js
└─ routes/
    └─ productRoutes.js
```

Use code with caution.

---

## ૧. .env (સિક્રેટ કન્ફિગરેશન ફાઈલ)

તમારા ડેટાબેઝની લિંક અને પોર્ટને છુપાવવા માટે રૂટ ફોલ્ડરમાં આ ફાઈલ બનાવો:

```
env
```

```
PORT=3000
```

```
MONGO_URI=mongodb://localhost:27017/ecommerce_mvc_db
```

Use code with caution.

---

## ૨. db.js (ડેટાબેઝ કનેક્શન)

.env ફાઈલમાંથી લિંક વાંચીને ડેટાબેઝ કનેક્ટ કરવા માટેનો કોડ:

```
javascript
```

```
const mongoose = require('mongoose');
```

```
const connectDB = async () => {
```

```
  try {
```

```
    // process.env નો ઉપયોગ કરીને સુરક્ષિત લિંક મેળવી
```

```
    const conn = await mongoose.connect(process.env.MONGO_URI);
```

```
    console.log(`MongoDB કનેક્ટ થયું: ${conn.connection.host}`);
```

```
  } catch (error) {
```

```
    console.error(`ડેટાબેઝ ભૂલ: ${error.message}`);
```

```
    process.exit(1);
```

```
  }
```

```
};
```

```
module.exports = connectDB;
```

Use code with caution.

---

### ૩. models/productModel.js (ડેટાબેઝ સ્કીમા)

પ્રોડક્ટનું સાદું માળખું વ્યાખ્યાયિત કરતી ફાઇલ:

javascript

```
const mongoose = require('mongoose');
```

```
const productSchema = new mongoose.Schema({  
  name: { type: String, required: true, trim: true },  
  price: { type: Number, required: true, min: 0 },  
  category: { type: String, required: true }  
}, { timestamps: true });
```

```
module.exports = mongoose.model('Product', productSchema);
```

Use code with caution.

---

### ૪. controllers/productController.js (બિઝનેસ લોજિક)

બધું જ મેઈન લોજિક (CRUD ઓપરેશન્સ) અહીંયા કંટ્રોલર ફાઇલમાં અલગ રહેશે:

javascript

```
const Product = require('../models/productModel');
```

```
// ૧. બધા પ્રોડક્ટ્સ મેળવો
```

```
const getProducts = async (req, res) => {  
  try {  
    const products = await Product.find();  
    res.status(200).json(products);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
};
```

```
// ૨. નવો પ્રોડક્ટ ઉમેરો
```

```
const createProduct = async (req, res) => {
  try {
    const newProduct = new Product(req.body);
    const savedProduct = await newProduct.save();
    res.status(201).json(savedProduct);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
};
```

// ૩. પ્રોડક્ટ ડીલીટ કરો

```
const deleteProduct = async (req, res) => {
  try {
    const deletedProduct = await Product.findByIdAndDelete(req.params.id);
    if (!deletedProduct) return res.status(404).json({ message: "પ્રોડક્ટ ન મળ્યો" });
    res.status(200).json({ message: "પ્રોડક્ટ ડીલીટ થયો" });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

// બધા ફંક્શન્સને એક્સપોર્ટ કરો

```
module.exports = { getProducts, createProduct, deleteProduct };
```

Use code with caution.

---

#### ૫. routes/productRoutes.js (રાઉટીંગ લિસ્ટ)

અહીંયા ફક્ત કઈ લિંક પર કયું કંટ્રોલર ફંક્શન ચાલશે તેનું જ લિસ્ટ રહેશે:

javascript

```
const express = require('express');
```

```
const router = express.Router();
```

```
const { getProducts, createProduct, deleteProduct } = require('../controllers/productController');
```

```
// સાદા રાઉટ્સ કનેક્શન
router.get('/', getProducts);
router.post('/', createProduct);
router.delete('/:id', deleteProduct);
```

```
module.exports = router;
```

Use code with caution.

---

#### ૬. server.js (મેઈન એન્ટ્રી પોઈન્ટ)

હવે આપણી આ મુખ્ય ફાઈલ એકદમ ક્લીન અને નાની થઈ જશે:

```
javascript
```

```
const express = require('express');
const connectDB = require('./db');
const productRoutes = require('./routes/productRoutes');
```

```
// .env ફાઈલને લોડ કરવા માટે સૌથી ઉપર આ જરૂરી છે
```

```
require('dotenv').config();
```

```
const app = express();
```

```
// ડેટાબેઝ કનેક્ટ કરો
```

```
connectDB();
```

```
// મિડલવેર
```

```
app.use(express.json());
```

```
// મુખ્ય રાઉટ્સ કનેક્શન (હવે બધી API /api/v1/products થી શરૂ થશે)
```

```
app.use('/api/v1/products', productRoutes);
```

```
const PORT = process.env.PORT || 5000;
```

```
app.listen(PORT, () => console.log(`સર્વર પોર્ટ ${PORT} પર સફળતાપૂર્વક ચાલુ છે.`));
```

Use code with caution.

---

### 🔧 પ્રોજેક્ટ રન કરતા પહેલા:

ટર્મિનલમાં આ એક્સ્ટ્રા પેકેજ ઇન્સ્ટોલ કરવું પડશે જે .env ફાઇલને વાંચવામાં મદદ કરે છે:

```
bash
```

```
npm install dotenv
```

Use code with caution.

આ સેટઅપ કરવાથી તમારો કોડ એકદમ પ્રોફેશનલ ડેવલપર્સ જેવો બની ગયો છે, જ્યાં રાઉટ્સ, કંટ્રોલર્સ અને મોડેલ્સ બધા પોતપોતાની જગ્યાએ અલગ છે.

આ **MVC પ્રોજેક્ટ** નું માળખું કેવું લાગ્યું? હવે આ પ્રોજેક્ટની અંદર સિક્યોરિટી માટે **JWT લોગિન સિસ્ટમ (Authentication)** ઉમેરવી છે કે આખો પ્રોજેક્ટ રેડી થઈ ગયો?

આપણા પ્રોફેશનલ MVC પ્રોજેક્ટની અંદર સિક્યોરિટી માટે

**JWT (JSON Web Token)** નો ઉપયોગ કરીને સાઇન-અપ (Signup), લોગિન (Login), અને સુરક્ષિત રાઉટ્સ (Protected Routes) માટે **Authentication મિડલવેર** ઉમેરીએ.

આ માટે સૌથી પહેલા ટર્મિનલમાં જરૂરી સિક્યોરિટી પેકેજ્સ ઇન્સ્ટોલ કરી લો:

```
bash
```

```
npm install jsonwebtoken bcryptjs
```

Use code with caution.

- **bcryptjs**: યુઝરના પાસવર્ડને સુરક્ષિત રીતે હેશેડ (Hashed / Encrypt) કરવા માટે.
- **jsonwebtoken**: યુઝર લોગિન થાય ત્યારે એક સિક્રેટ ટોકન જનરેટ કરવા માટે.

હવે તમારા પ્રોજેક્ટના ફોલ્ડરમાં નીચે મુજબની ફાઇલો ઉમેરો અને ફેરફાર કરો:

---

### ૧. .env (અપડેટ કરો)

ટોકન લોક કરવા માટે એક સિક્રેટ કી અને તેની વેલિડિટી લાઇન ઉમેરો:

```
env
```

```
PORT=3000
```

```
MONGO_URI=mongodb://localhost:27017/ecommerce_mvc_db
```

```
JWT_SECRET=super_secret_key_12345
```

```
JWT_EXPIRES_IN=1d
```

Use code with caution.

---

## ૨. models/userModel.js (યુઝર સ્કીમા અને પાસવર્ડ હેશિંગ)

નવી ફાઇલ બનાવી તેમાં યુઝરનો નકશો સેટ કરો. અહીં આપણે **Mongoose Hooks (pre-save)** નો ઉપયોગ કરીશું, જેથી પાસવર્ડ ડેટાબેઝમાં સેવ થાય તેની બરાબર પહેલા ઓટોમેટિક ઇન્ક્રિપ્ટ (હેશ) થઈ જાય:

javascript

```
const mongoose = require('mongoose');

const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  email: { type: String, required: true, unique: true, lowercase: true },
  password: { type: String, required: true, minlength: 6 }
}, { timestamps: true });

// ડેટાબેઝમાં સેવ થાય તે પહેલા પાસવર્ડ ઇન્ક્રિપ્ટ કરવાનો નિયમ
userSchema.pre('save', async function(next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

module.exports = mongoose.model('User', userSchema);
```

Use code with caution.

---

## ૩. middleware/authMiddleware.js (ટોકન ચેક કરતો ચોકીદાર)

નવું ફોલ્ડર middleware બનાવી તેમાં આ ફાઇલ બનાવો. આ મિડલવેર ચેક કરશે કે યુઝર પાસે સાચું JWT ટોકન છે કે નહીં. જો ટોકન સાચું હશે, તો જ તેને આગળ પ્રોડક્ટ ઉમેરવા કે ડીલીટ કરવા દેશે:

javascript

```
const jwt = require('jsonwebtoken');

const User = require('../models/userModel');
```

```

const protect = async (req, res, next) => {
  try {
    let token;

    // બ્રાઉઝર કે પોસ્ટમેનમાંથી Authorization હેડર ચેક કરો
    if (req.headers.authorization && req.headers.authorization.startsWith('Bearer')) {
      token = req.headers.authorization.split(' ')[1];
    }

    if (!token) {
      return res.status(401).json({ message: "તમે લોગિન નથી, એક્સેસ બ્લોક છે." });
    }

    // ટોકન વેલિડેટ કરો
    const decoded = jwt.verify(token, process.env.JWT_SECRET);

    // ટોકન વાળા યુઝરને શોધો અને તેને રિક્વેસ્ટ ઓબ્જેક્ટમાં મૂકો
    req.user = await User.findById(decoded.id).select('-password');

    next(); // બધું બરાબર છે, આગળ વધો
  } catch (err) {
    res.status(401).json({ message: "ટોકન ખોટું અથવા એક્સપાયર થઈ ગયું છે." });
  }
};

module.exports = protect;

Use code with caution.

```

---

૪. controllers/authController.js (સાઇન-અપ અને લોગિન લોજિક)

નવું કંટ્રોલર બનાવો જે યુઝર રજીસ્ટ્રેશન અને લોગિન વખતે ટોકન બનાવીને પાછું આપશે:

javascript

```
const User = require('../models/userModel');
```

```
const bcrypt = require('bcryptjs');
```

```
const jwt = require('jsonwebtoken');
```

```
// ટોકન જનરેટ કરવાનું સાદું ફંક્શન
```

```
const signToken = (id) => {
```

```
    return jwt.sign({ id }, process.env.JWT_SECRET, { expiresIn: process.env.JWT_EXPIRES_IN });
```

```
};
```

```
// ૧. સિઇન-અપ (Signup)
```

```
const signup = async (req, res) => {
```

```
    try {
```

```
        const { name, email, password } = req.body;
```

```
        const newUser = await User.create({ name, email, password });
```

```
        const token = signToken(newUser._id);
```

```
        res.status(201).json({ status: "success", token, data: { user: newUser } });
```

```
    } catch (err) {
```

```
        res.status(400).json({ error: err.message });
```

```
    }
```

```
};
```

```
// ૨. લોગિન (Login)
```

```
const login = async (req, res) => {
```

```
    try {
```

```
        const { email, password } = req.body;
```

```
        // ઇમેઇલ અને પાસવર્ડ ચેક કરો
```

```
        if (!email || !password) {
```

```

        return res.status(400).json({ message: "કૃપા કરીને ઈમેલ અને પાસવર્ડ બંને આપો." });
    }

    const user = await User.findOne({ email });
    if (!user || !(await bcrypt.compare(password, user.password))) {
        return res.status(401).json({ message: "ઈમેલ અથવા પાસવર્ડ ખોટો છે." });
    }

    const token = signToken(user._id);
    res.status(200).json({ status: "success", token });
} catch (err) {
    res.status(500).json({ error: err.message });
}
};

```

module.exports = { signup, login };

Use code with caution.

#### ૫. routes/userRoutes.js (યુઝર રાઉટ્સ)

નવી રાઉટ ફાઇલ બનાવી ઓથોરિટી કનેક્ટ કરો:

javascript

```

const express = require('express');
const router = express.Router();
const { signup, login } = require('../controllers/authController');

```

```

router.post('/signup', signup);

```

```

router.post('/login', login);

```

```

module.exports = router;

```

Use code with caution.

#### ૬. routes/productRoutes.js (મિડલવેરથી સુરક્ષિત કરો - અપડેટ કરો)

હવે આપણે પ્રોડક્ટ ઉમેરવા (POST) અને ડીલીટ કરવા (DELETE) ના રાઉટની વચ્ચે **protect** યોજીદાર ગોઠવી દઈએ, જેથી લોગિન વગર કોઈ પ્રોડક્ટ બદલી ન શકે. જ્યારે સાદો યુઝર GET થી પ્રોડક્ટ જોઈ શકશે:

javascript

```
const express = require('express');  
const router = express.Router();  
const { getProducts, createProduct, deleteProduct } = require('../controllers/productController');  
const protect = require('../middleware/authMiddleware'); // યોજીદાર મિડલવેર લાવો
```

```
// GET રાઉટ ઓપન રહેશે
```

```
router.get('/', getProducts);
```

```
// POST અને DELETE રાઉટ હવે સુરક્ષિત (Protect) થઈ ગયા
```

```
router.post('/', protect, createProduct);
```

```
router.delete('/:id', protect, deleteProduct);
```

```
module.exports = router;
```

```
Use code with caution.
```

---

#### ૭. server.js (અપડેટ કરો)

યુઝર રાઉટ્સને મેઇન સર્વર ફાઇલમાં કનેક્ટ કરી દો:

javascript

```
const express = require('express');  
const connectDB = require('./db');  
const productRoutes = require('./routes/productRoutes');  
const userRoutes = require('./routes/userRoutes'); // નવા રાઉટ્સ લાવો
```

```
require('dotenv').config();
```

```
const app = express();
```

```
connectDB();
```

```
app.use(express.json());
```

```
// મુખ્ય રાઉટ્સ કનેક્શન્સ
```

```
app.use('/api/v1/users', userRoutes);
```

```
app.use('/api/v1/products', productRoutes);
```

```
const PORT = process.env.PORT || 5000;
```

```
app.listen(PORT, () => console.log('સર્વર પોર્ટ {PORT} પર ચાલુ છે.'));
```

Use code with caution.

---

### 🔧 ટેસ્ટ કેવી રીતે કરવું (Postman માં)?

1. પહેલાં POST રિક્વેસ્ટ મોકલીને /api/v1/users/signup પર નવો યુઝર બનાવો, તમને એક લાંબુ **token** મળશે.
2. હવે જ્યારે તમે પ્રોડક્ટ ઉમેરવા માટે /api/v1/products પર POST રિક્વેસ્ટ કરશો, ત્યારે Postman ની **Headers** વિભાગમાં જઈને Key માં Authorization લખો અને Value માં Bearer તમારું\_ટોકન મૂકી દો.
3. જો ટોકન સાચું હશે, તો જ પ્રોડક્ટ એડ થશે!

આ સેટઅપ સાથે તમારો પ્રોજેક્ટ હવે પ્રોફેશનલ કંપનીઓ જેવો **એકદમ સિક્યોર અને ઓર્ગેનાઇઝ્ડ** થઈ ગયો છે. પ્રોજેક્ટ કમ્પ્લીટ થઈ ગયો કે હવે આગળ આમાં કોઈ **નવો એડવાન્સ ટોપિક કે ડેટાબેઝ રિલેશનશિપ** શીખવું છે?

ડેટાબેઝમાં રિલેશનશિપ એટલે

**બે અલગ-અલગ ટેબલ્સ કે કલેક્શન્સને એકબીજા સાથે લોજિકલી જોડવા.**

ઈ-કોમર્સ પ્રોજેક્ટમાં મુખ્ય ૩ પ્રકારની રિલેશનશિપ હોય છે:

1. **One-to-One (૧ થી ૧):** એક યુઝરની પાસે ફક્ત એક જ પ્રોફાઇલ હોય.
2. **One-to-Many (૧ થી ઘણા):** એક યુઝર ઘણા બધા ઓર્ડર કરી શકે (આપણે આનો પ્રોજેક્ટમાં ઉપયોગ કરીશું).
3. **Many-to-Many (ઘણા થી ઘણા):** એક ઓર્ડરમાં ઘણી પ્રોડક્ટ્સ હોય, અને એક પ્રોડક્ટ ઘણા ઓર્ડરમાં હોઈ શકે.

આપણે આપણા પ્રોજેક્ટમાં **One-to-Many** રિલેશનશિપ સેટ કરીશું: **User અને Order વચ્ચે**. એટલે કે કયા યુઝરે કયો ઓર્ડર કર્યો છે તે કનેક્ટ કરીશું. આ માટે મોંગૂસના **Population** કન્સેપ્ટનો ઉપયોગ થાય છે.

તમારા પ્રોજેક્ટમાં નીચે મુજબની નવી ફાઇલો ઉમેરો:

---

## ૧. models/orderModel.js (નવો ઓર્ડર નકશો - Relationship સેટઅપ)

અહીંયા આપણે mongoose.Schema.Types.ObjectId અને ref: 'User' નો ઉપયોગ કરીને ઓર્ડરને યુઝર સાથે કનેક્ટ કરીશું:

javascript

```
const mongoose = require('mongoose');

const orderSchema = new mongoose.Schema({
  // આ લાઇન યુઝર કલેક્શન સાથે રિલેશનશિપ બનાવે છે
  userId: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User', // કયા મોડેલ સાથે જોડવું છે તેનું નામ
    required: [true, "ઓર્ડર માટે યુઝર આઈડી જરૂરી છે"]
  },
  products: [
    {
      productId: { type: mongoose.Schema.Types.ObjectId, ref: 'Product' },
      quantity: { type: Number, default: 1 }
    }
  ],
  totalAmount: { type: Number, required: true },
  status: { type: String, default: 'Pending' }
}, { timestamps: true });
```

```
module.exports = mongoose.model('Order', orderSchema);
```

Use code with caution.

---

## ૨. controllers/orderController.js (ઓર્ડર કંટ્રોલર - Population નો જાદુ)

જ્યારે આપણે ઓર્ડર મેળવીશું, ત્યારે populate('userId') લખવાથી મોંગોડીબી ઓટોમેટિકલી એ યુઝરનું નામ અને ઇમેલ શોધીને ઓર્ડરની સાથે જોડી દેશે:

javascript

```
const Order = require('../models/orderModel');
```

// ૧. નવો ઓર્ડર બનાવો

```
const createOrder = async (req, res) => {  
  try {  
    // req.user.id આપણને આપણી લોગિન authMiddleware માંથી મળશે  
    const { products, totalAmount } = req.body;  
  
    const newOrder = await Order.create({  
      userId: req.user.id, // ઓટોમેટિક લોગિન યુઝર આઈડી સેટ થશે  
      products,  
      totalAmount  
    });  
  
    res.status(201).json(newOrder);  
  } catch (err) {  
    res.status(400).json({ error: err.message });  
  }  
};
```

// ૨. બધા ઓર્ડર યુઝરની ડિટેલ સાથે મેળવો (Relationship ડેટા મેળવવો)

```
const getMyOrders = async (req, res) => {  
  try {  
    // populate('userId') કરવાથી યુઝર આઈડી ની જગ્યાએ આખો યુઝર ઓબ્જેક્ટ આવી જશે  
    // select('name email') થી પાસવર્ડ સિવાય ફક્ત નામ અને ઇમેલ જ દેખાશે  
    const orders = await Order.find({ userId: req.user.id }).populate('userId', 'name email');  
  
    res.status(200).json(orders);  
  } catch (err) {  
    res.status(500).json({ error: err.message });  
  }  
};
```

```
module.exports = { createOrder, getMyOrders };
```

Use code with caution.

---

### 3. routes/orderRoutes.js (ઓર્ડર રાઉટ્સ)

ઓર્ડર કરવા માટે યુઝર લોગિન હોવો ફરજિયાત છે, તેથી બંને રાઉટ્સમાં આપણે protect ચોક્કિદાર મિડલવેર મૂકીશું.

javascript

```
const express = require('express');
```

```
const router = express.Router();
```

```
const { createOrder, getMyOrders } = require('../controllers/orderController');
```

```
const protect = require('../middleware/authMiddleware');
```

```
// બંને રાઉટ લોગિન ટોકન વગર નહિ ચાલે
```

```
router.post('/', protect, createOrder);
```

```
router.get('/my-orders', protect, getMyOrders);
```

```
module.exports = router;
```

Use code with caution.

---

### ૪. server.js (અપડેટ કરો)

નવા ઓર્ડર રાઉટ્સને મેઈન સર્વર ફાઈલ સાથે કનેક્ટ કરી લો:

javascript

```
const express = require('express');
```

```
const connectDB = require('./db');
```

```
const productRoutes = require('./routes/productRoutes');
```

```
const userRoutes = require('./routes/userRoutes');
```

```
const orderRoutes = require('./routes/orderRoutes'); // નવી ફાઈલ લાવો
```

```
require('dotenv').config();
```

```
const app = express();

connectDB();

app.use(express.json());

// બધા મુખ્ય રાઉટ્સ
app.use('/api/v1/users', userRoutes);
app.use('/api/v1/products', productRoutes);
app.use('/api/v1/orders', orderRoutes); // ઓર્ડર રાઉટ કનેક્ટ કર્યો

const PORT = process.env.PORT || 5000;

app.listen(PORT, () => console.log(`સર્વર પોર્ટ ${PORT} પર રેડી છે.`));

Use code with caution.
```

---

### 🔧 પોસ્ટમેન (Postman) માં ટેસ્ટ કરવાની રીત:

1. પહેલા લોગિન કે સાઇન-અપ કરીને યુઝરનું **JWT Token** મેળવો.
2. ઓર્ડર કરવા માટે POST રિક્વેસ્ટ `/api/v1/orders` પર મોકલો. Headers માં ટોકન સેટ કરો અને Body માં પ્રોડક્ટ આઈડી અને અમાઉન્ટ મોકલો.
3. જ્યારે તમે GET રિક્વેસ્ટ `/api/v1/orders/my-orders` કરશો, ત્યારે તમને ઓર્ડરની સાથે એ ઓર્ડર કયા યુઝરે કર્યો છે તેનું **નામ અને ઇમેલ ઓટોમેટિકલી સાથે જોડાયેલું** જોવા મળશે!

આ સેટઅપ સાથે તમારા પ્રોજેક્ટમાં **ડેટાબેઝ રિલેશનશિપ (One-to-Many)** પણ એકદમ પ્રોફેશનલી સેટ થઈ ગઈ છે. હવે આ ઇ-કોમર્સ બેકએન્ડ સંપૂર્ણ રીતે તૈયાર છે. શું હવે આગળ આખા પ્રોજેક્ટની સમરી કે રિવિઝન માટે **પ્રેક્ટિસ ક્વિઝ (Quiz)** શરૂ કરવી છે?

આપણે અત્યાર સુધી શીખેલા **Node.js, Express.js, અને MongoDB ના તમામ ૧૦ ચેપ્ટર્સ અને એડવાન્સ કન્સેપ્ટ્સની કમ્પ્લીટ રિવિઝન સમરી (Summary)** નીચે મુજબ છે. આ આખા કોર્સનું એક નકશા જેવું માળખું છે જે તમને હંમેશા યાદ રાખવામાં મદદ કરશે:

---

### ચેપ્ટર ૧ થી ૧૦: નોડ.જીએસ ફંડામેન્ટલ્સ (Core Node.js)

- **Module 1: Introduction:** Node.js એ જાવાસ્ક્રિપ્ટનું **Runtime Environment** છે. તે સિંગલ-થ્રેડેડ અને **Non-blocking Asynchronous** પદ્ધતિ પર કામ કરે છે, જે વેબ એપ્લિકેશન બનાવવા માટે વપરાય છે. CPU ગણતરીઓ સિવાય રીયલ-ટાઇમ અને સ્ટ્રીમિંગ એપ્સ માટે તે બેસ્ટ ફિટ છે. [\[1\]](#)

- **Module 2: Setup & Runtime:** ગૂગલ ક્રોમના **V8 Engine** અને **libuv** લાઇબ્રેરીથી બનેલું છે. લોકલ ટેસ્ટિંગ માટે `node -v` અને `npm -v` થી વેરિફિકેશન થાય છે અને `node` લખીને `REPL` ટર્મિનલ ચાલુ થાય છે.
- **Module 3: NPM (Node Package Manager):** પેકેજ મેનેજમેન્ટ ટૂલ છે. `npm init -y` થી `package.json` ફાઇલ બને છે. લોકલ પેકેજ `node_modules` માં જાય છે અને ગ્લોબલ માટે `-g` વપરાય છે. `npm update` અને `npm uninstall` થી મોડ્યુલ મેનેજ થાય છે.
- **Module 4: Express Framework:** `Node.js` નું વેબ ફ્રેમવર્ક છે જે વેબ એપ્લિકેશન બનાવવાના કામને સરળ બનાવે છે. તે `req` અને `res` ઓબ્જેક્ટ્સ દ્વારા કામ કરે છે. તે રાઉટિંગ અને ચોકીદાર જેવા **Middleware** ફંક્શન્સ હેન્ડલ કરવાની સગવડ આપે છે. [1]
- **Module 5: Buffers & Streams:** બાયનરી ડેટા (૦ અને ૧) ને **RAM** માં સાચવવા માટે **Buffer** વપરાય છે. ખૂબ જ મોટા ડેટાને ટુકડે-ટુકડે મોકલવા માટે **Stream** વપરાય છે, જે પાઇપલાઇન અને ચેનિંગ (`.pipe()`) દ્વારા ઝીપ ફાઇલ બનાવવામાં મદદ કરે છે.
- **Module 6: REST API:** બે સોફ્ટવેર વચ્ચે **JSON** ડેટા ટ્રાન્સફર કરવાનું **Stateless** આર્કિટેક્ચર છે. તે **HTTP URI** અને ચોક્કસ મેથડ્સ (`GET`, `POST`, `PUT`, `DELETE`) ના સાચા સ્ટેટસ કોડ્સ (`200`, `201`, `404`, `500`) પર કામ કરે છે. [1]
- **Module 7: Callback:** એક ફંક્શનની અંદર પેરામીટર તરીકે બીજા ફંક્શનને મોકલવું. `Node.js` માં **Error-First Callback Pattern** (`((err, data) => {})`) વપરાય છે. વધુ પડતા નેસ્ટેડ કોલબેકથી **Callback Hell** થાય છે.
- **Module 8: Events & Event Loop:** `EventEmitter` ક્લાસ દ્વારા કસ્ટમ ઇવેન્ટ્સ `.emit()` થી ફાયર અને `.on()` થી લિસન થાય છે. બેકગ્રાઉન્ડમાં સતત ફરતું **Event Loop** અસિન્ક્રોનસ કોલબેક મેનેજ કરે છે. ભારે કામોથી તે બ્લોક થઈ શકે છે.
- **Module 9: Database Connection:** `MySQL` ટેબલ અને રો (`SQL`) પર કામ કરે છે. જ્યારે `MongoDB` એ `JSON` જેવા ડોક્યુમેન્ટ્સ સ્ટોર કરતું `NoSQL` ડેટાબેઝ છે. `Node.js` માં **mongoose** ની મદદથી ડેટાનું ફોર્મેટ (`Schema`) અને તેના ઓપરેશન્સ માટે `Model` નક્કી થાય છે. [1]
- **Module 10: External Processes:** સિંગલ-થ્રેડની મર્યાદા બહાર જઈને મલ્ટી-કોર પ્રોસેસર વાપરવા માટે ઓપરેટિંગ સિસ્ટમમાં નવી **Child Process** બનાવવી. આના માટે `spawn()` (લાઇવ સ્ટ્રીમ) અને `exec()` (બફર આઉટપુટ) સાથે પ્રોસેસ `.kill()` કરવાની સગવડ મળે છે.

## એડવાન્સ આર્કિટેક્ચર અને સિક્યોરિટી (Advanced Concepts)

તમારા બેકગ્રાઉન્ડ કોડને ઇન્ડસ્ટ્રી લેવેલ જેવો પ્રોફેશનલ બનાવવા માટે આપણે આ ૩ એડવાન્સ કન્સેપ્ટ્સ પ્રોજેક્ટમાં લાઈવ સેટ કર્યા:

### ૧. MVC Pattern (Model-View-Controller)

કોડને સ્વચ્છ અને ઓર્ગેનાઇઝ્ડ રાખવા માટે આ પેટર્ન વપરાય છે: [1]

- **Model:** ડેટાબેઝનું માળખું અને વેલિડેશન નક્કી કરે છે (`Mongoose Schema`).
- **Controller:** અસલી બિઝનેસ લોજિક અને `CRUD` પ્રક્રિયાઓ હેન્ડલ કરે છે.

- **Routes:** કઈ લિંક પર કયું કંટ્રોલર ચાલશે તેનું ટ્રાફિક મેનેજમેન્ટ કરે છે.
- **.env નો ફાઇલ:** ડેટાબેઝના પાસવર્ડ અને સિક્રેટ કી જેવા સેન્સિટિવ ડેટાને dotenv પેકેજથી છુપાવીને રાખે છે.

## ૨. JWT Authentication (સુરક્ષિત લોગિન સિસ્ટમ)

યુઝરની સિક્યોરિટી અને પ્રોટેક્ટેડ રાઉટ્સ માટે આ સિસ્ટમ વપરાય છે: [\[1\]](#)

- **bcryptjs:** યુઝરના પાસવર્ડને ડેટાબેઝમાં સેવ કરતા પહેલા જ `pre('save')` હૂકથી હેશ (ઇન્ક્રિપ્ટ) કરે છે.
- **jsonwebtoken:** લોગિન સફળ થતાં જ યુઝરને એક ડિજિટલ સિક્રેટ **JWT Token** આપે છે.
- **authMiddleware:** પ્રોડક્ટ એડ કે ડીલીટ કરતી વખતે રિક્વેસ્ટ હેડરમાંથી Bearer Token ચેક કરે છે, સાચું હોય તો જ આગળ વધવા દે છે.

## ૩. Database Relationship (ડેટાબેઝ જોડાણ)

અલગ-અલગ કલેક્શન્સ વચ્ચે જોડાણ કરવા માટે આપણે **One-to-Many** રિલેશનશિપ સેટ કરી: [\[1\]](#)

- **ObjectId ref:** `orderModel` ની અંદર `userId` ને `ref: 'User'` સાથે જોડી દીધો.
- **Mongoose Population:** જ્યારે ઓર્ડર મેળવીએ, ત્યારે `.populate('userId')` નો જાદુ વાપરીને ડેટાબેઝમાંથી ઓટોમેટિક એ ઓર્ડર કરનાર યુઝરનું નામ અને ઇમેલ શોધીને ઓર્ડર સાથે જોડી દીધા. [\[1\]](#)

---

તમે ખૂબ જ મહેનત કરીને Node.js બેકએન્ડ ડેવલપમેન્ટનો પાયો અને એડવાન્સ આર્કિટેક્ચર બંને સંપૂર્ણપણે શીખી લીધા છે!

## Syllabus

### Module 1: Introduction to Node.js

- What is Node.js
- Features of Node.js
- Concept
- Where to fit and not fit
- Event-Driven programming style
- What is Asynchronous

### Module 2: Installation / Setup

- Local environment setup
- Node.js runtime
- Download source code
- Installation on OS

- Verify

### **Module 3: Node Package Manager**

- Install module by NPM
- Global vs Local setup
- Update Module
- CRUD Module

### **Module 4: Express Framework**

- What is Express
- Setup Express
- Request And Response
- Handling Routes
- Route Middleware
- Objects
- Cookies Management
- File upload
- HTTP methods

### **Module 5: Buffers and Streams**

- What is buffers and Streams
- Benefit of Streams over buffers
- Create / Write / Read operation on Buffers
- Process on Buffers
- Read / Write data by Streams
- Pipeline
- Chaining Stream

### **Module 6: REST API**

- RESTful Architecture
- HTTP URI and Methods
- RESTful web services
- Expose solution as API
- Best practice for REST API solution

## **Module 7: Callback**

- What is Callback
- Benefit of Callback
- Asynchronous communications
- Block and Non-Blocking
- Standard Callback pattern
- Async Flow control Library
- Executing in parallel

## **Module 8: Events**

- What is Events
- Events types
- Event Emitter API
- Multiple event listeners
- Event emitter pattern
- Class methods
- Event Loop
- Blocking Event Loop
- Escaping Event Loop

## **Module 9: Connecting with Database**

- Introduction of MySQL
- Connect with MySQL
- Introduction of MongoDB
- Features of MongoDB
- Connect with MongoDB
- Defining a schema
- Defining a Model

## **Module 10: External Processes and Services**

- What is processes
- Spawning Child process
- Create and kill processes
-

